
单片机动态加载

基于 STM32H743

王尊恒

哈尔滨工程大学

目 录

目 录.....	1
前 言.....	2
第一部分 背景知识.....	3
1.1 MDK 编译过程简介.....	3
1.2 程序组成、存储与运行.....	4
1.2.1 CODE、RO、RW、ZI Data 域及堆栈空间.....	4
1.2.2 程序的存储与运行.....	5
1.3 ELF 文件简介.....	6
1.3.1 ELF 文件格式.....	8
第二部分 动态加载器实现思路与环境搭建.....	10
2.1 什么是动态加载？.....	10
2.2 fromelf 工具使用.....	11
2.2.1 环境变量配置.....	11
2.2.2 fromelf 工具简单示例.....	13
第三部分 生成具有重定位节区的.axf 文件.....	17
3.1 输出具有重定位节区的.axf 工程搭建.....	17
3.2 工程搭建的原理.....	24
3.3 分散加载文件编写依据.....	26
3.4 小结.....	26
第四部分 动态加载器的实现.....	27
4.1 动态加载器 API 介绍.....	27
4.2 动态加载器 API 实现细节.....	30
4.3.axf 文件调用被加载器函数的方法介绍.....	39
4.3.1 通过固定的函数表访问.....	39
4.3.2 通过特定的指令访问.....	40

前 言

本文主要介绍如何在单片机上实现动态加载功能，需要读者具有一定的嵌入式开发基础且已经可以熟练运用 MDK 创建应用程序。其中所讲的生成具有动态段的 elf 文件方法有一部分仅适用于 MDK 平台 ARM clang 编译器，因此推荐读者也使用与作者一样的软件平台。本文的主要参考资料为《Arm Compiler User Guide Version 6.16 (PDF)》、ARM 官网中关于动态链接的介绍

<https://developer.arm.com/documentation/ihl0044/h/?lang=en#dynamic-linking>、《ELF 手册-中文版.pdf》、《野火 HAL 库开发指南》第 49 章、《安富莱_STM32-V7 开发板_用户手册》等。本人写作能力有限，如果有哪个地方没讲清楚的，可以在安富莱电子论坛下的帖子给我留言，论坛帖子地址：

<https://www.armbbs.cn/forum.php?mod=viewthread&tid=109952&fromuid=24016>

第一部分 背景知识

本节主要介绍实现动态加载的背景知识，比较重要。

1.1 MDK 编译过程简介

MDK 编译过程与大部分的编译器都是类似的，图 1.1 展示了 MDK 的大致编译过程。

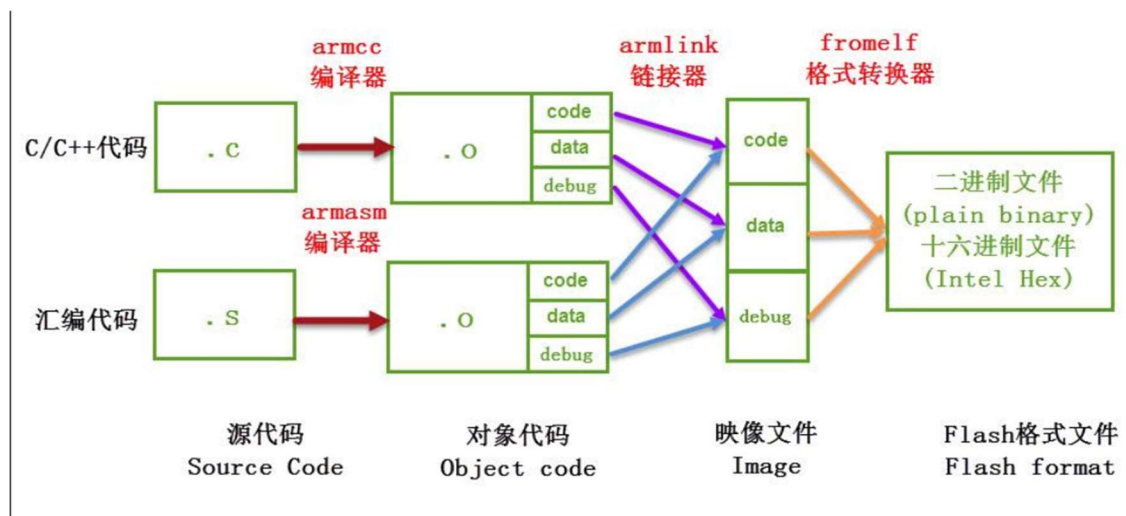


图 1.1 MDK 的大致编译过程

此处出现的文件类型在后面将会详细介绍，现在先捋一下编译输出的大致过程：

- （1）编译：MDK 通过 armcc&armclang(编译器)和 armasm(汇编器)将源代码文件(由 C 语言、C++语言、汇编语言编写的文件)编译为后缀名为.o 的对象文件(Object Code，也称目标文件)，其内容主要包括源代码文件对应的机器码，包含代码、数据、调试信息等。
- （2）链接：链接器 armlink 把各个.o 文件及库文件链接成一个映像文件 “.axf” 或 “.elf” 。
- （3）格式转换：一般来说 Windows 或 Linux 系统使用链接器直接生成可执行映像文件 elf 后，内核根据该文件的信息加载后，就可以运行程序了，但在单片机平台上，需要该文件的内容加载到芯片上，所以还需要对链接器生成的 elf 映像文件利用格式转换器 fromelf 转换成 “.bin” 或 “.hex” 文件，交给下载器下载到芯片的 FLASH 或 ROM 中。一般我们用 MDK 界面的下载键  进行下载时，加载的就是编译输出目录中的 “.axf” 文件。

1.2 程序组成、存储与运行

1.2.1 CODE、RO、RW、ZI Data 域及堆栈空间

```
compiling stm32h7xx_hal_spi.c...
compiling stm32h7xx_ll_delayblock.c...
compiling stm32h7xx_hal_tim_ex.c...
compiling stm32h7xx_hal_uart.c...
compiling stm32h7xx_hal_uart.c...
compiling stm32h7xx_ll_fsmc.c...
assembling startup_stm32h743xx.s...
compiling stm32h7xx_ll_sdmmc.c...
compiling system_stm32h7xx.c...
compiling stm32h7xx_hal_tim.c...
compiling USB_Config_0.c...
compiling stm32h7xx_ll_usb.c...
linking...
Program Size: Code=627308 RO-data=128412 RW-data=5216 ZI-data=28306256
"..\\OBJ\\STM32H7_RT5.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:32
```

图 1.2 MDK 的编译输出

在工程的编译提示输出信息中有一个语句“Program Size: Code=xx RO-data=xx RWdata=xx ZI-data=xx”，它说明了程序各个域的大小，编译后，应用程序中所有具有同一性质的数据(包括代码)被归到一个域，程序在存储或运行的时候，不同的域会呈现不同的状态，这些域的意义如下：

(1)Code: 即代码域，它指的是编译器生成的机器指令，这些内容被存储到 ROM 区。

(2) RO-data: Read Only data，即只读数据域，它指程序中用到的只读数据，这些数据被存储在 ROM 区，因而程序不能修改其内容。例如 C 语言中 `const` 关键字定义的变量就是典型的 RO-data。

(3) RW-data: Read Write data，即可读写数据域，它指初始化为“非 0 值”的可读写

数据，程序刚运行时，这些数据具有非 0 的初始值，且运行的时候它们会常驻在 RAM 区，因而应用程序可以修改其内容。例如 C 语言中使用定义的全局变量，且定义时赋予“非 0 值”给该变量进行初始化。

(4) ZI-data: Zero Initialie data，即 0 初始化数据，它指初始化为“0 值”的可读写数据域，它与 RW-data 的区别是程序刚运行时这些数据初始值全都为 0，而后续运行过程与 RW-data 的性质一样，它们也常驻在 RAM 区，因而应用程序可以更改其内容。例如 C 语言中使用定义的全局变量，且定义时赋予“0 值”给该变量进行初始化

(若定义该变量时没有赋予初始值，编译器会把它当 ZI-data 来对待，由链接器在 main 函数前插入一段代码，使之初始化为 0.)

(5) ZI-data 的栈空间(Stack)及堆空间(Heap): 在 C 语言中，函数内部定义的局部变量属于栈空间，进入函数的时候从向栈空间申请内存给局部变量，退出时释放局部变量，归还内存空间。而使用 malloc 动态分配的变量属于堆空间。在程序中的栈空间和堆空间都是属于 ZI-data 区域的，这些空间都会被初始化为 0 值。编译器给出的 ZI-data 占用的空间值中包含了堆栈的大小(其实在 arm 中，堆空间并不完全是 malloc 函数管理的，有一小部分会作为 stdio 的缓冲区，感兴趣的可以查看《Arm C and C++ Libraries and Floating-Point Support User Guide》这个 pdf 进行学习)。

程序组件	所属类别
机器代码指令	Code
常量	RO-data
初值非 0 的全局变量	RW-data
初值为 0 的全局变量	ZI-data
局部变量	ZI-data 栈空间
使用 malloc 动态分配的空间	ZI-data 堆空间

1.2.2 程序的存储与运行

RW-data 和 ZI-data 它们仅仅是初始值不一样而已，为什么编译器非要把它们区分开？这就涉及到程序的存储状态了，应用程序具有静止状态和运行状态。静止态的程序被存储在非易失存储器中，如 STM32 的内部 FLASH，因而系统掉电后也能正常保存。但是当程序在运行状态的时候，程序常常需要修改一些暂存数据，由于运行速度的要求，这些数据往往存放在内存中(RAM)，掉电后这些数据会丢失。因此，程序在静止与运行

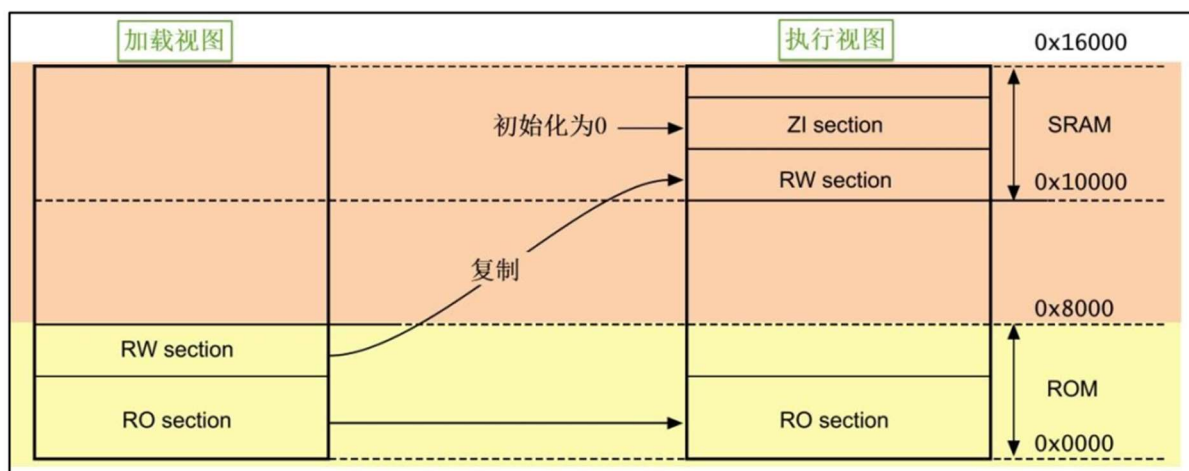


图 1.3 应用程序的加载视图与执行视图

的时候它在存储器中的表现是不一样的，见图 1.3。

图中的左侧是应用程序的存储状态，右侧是运行状态，而上方是 RAM 存储器区域，下方是 ROM 存储器区域。程序在存储状态时，RO 节(RO section)及 RW 节都被保存在 ROM 区。当程序开始运行时，内核直接从 ROM 中读取代码，并且在执行主体代码前，会先执行一段加载代码，它把 RW 节数据从 ROM 复制到 RAM，并且在 RAM 加入 ZI 节，ZI 节的数据都被初始化为 0。加载完后 RAM 区准备完毕，正式开始执行主体程序。编译生成的 RW-data 的数据属于图中的 RW 节，ZI-data 的数据属于图中的 ZI 节。是否需要掉电保存，这就是把 RW-data 与 ZI-data 区别开来的原因，因为在 RAM 创建数据的时候，默认值为 0，但如果有的数据要求初值非 0，那就需要使用 ROM 记录该初始值，运行时再复制到 RAM。STM32 的 RO 区域不需要加载到 SRAM，内核直接从 FLASH 读取指令运行。计算机系统的应用程序运行过程很类似，不过计算机系统的程序在存储状态时位于硬盘，执行的时候甚至会把上述的 RO 区域(代码、只读数据)加载到内存，加快运行速度，还有虚拟内存管理单元(MMU)辅助加载数据，使得可以运行比物理内存还大的应用程序。而 STM32 没有 MMU，所以无法支持 Linux 和 Windows 系统。

当程序存储到 STM32 芯片的内部 FLASH 时(即 ROM 区)，它占用的空间是 Code、RO-data 及 RW-data 的总和，所以如果这些内容比 STM32 芯片的 FLASH 空间大，程序就无法被正常保存了。当程序在执行的时候，需要占用内部 SRAM 空间(即 RAM 区)，占用的空间包括 RW-data 和 ZI-data。

在 MDK 中，我们建立的工程一般会选择芯片型号，选择后就有确定的 FLASH 及 SRAM 大小，若代码超出了芯片的存储器的极限，编译器会提示错误，这时就需要裁剪程序了，裁剪时可针对超出的区域来优化。

程序状态与区域	组成
程序执行时的只读区域(RO)	Code + RO data
程序执行时的可读写区域(RW)	RW data + ZI data
程序存储时占用的 ROM 区	Code + RO data + RW data

1.3 ELF 文件简介

可执行链接格式(elf)是 UNIX 系统实验室(USL)作为应用程序二进制接口(Application Binary Interface(ABI)而开发和发布的。工具接口标准委员会(TIS)选

择了正在发展中的 ELF 标准作为工作在 32 位 INTEL 体系上不同操作系统之间可移植的二进制文件格式。在 MDK 中编译输出的 .axf 文件与 .o 文件均为 ELF 文件。

Elf 文件有三个种类：

1. 可重定位的对象文件(Relocatable file)

这是由编译器和汇编器汇编生成的 .o 文件。后面的链接器(link editor)拿一个或一些可重定位文件作为输入，经链接处理后，生成一个可执行的对象文件(Executable file) 或者一个可被共享的对象文件(Shared object file)。我们可以使用 MDK 中 armlib 工具将众多的 .o 文件归档(archive)成 .lib 静态库文件。

2. 可执行的对象文件(Executable file)

像电脑中的 .exe 文件，下载到单片机的 axf 文件都属于此类。

3. 可被共享的对象文件(Shared object file)

这些就是所谓的动态库文件，也即 .so/.dll 文件。如果拿前面的静态库来生成可执行程序，那每个生成的可执行程序中都会有一份库代码的拷贝。如果在磁盘中存储这些可执行程序，那就会占用额外的磁盘空间；另外如果拿它们放到 Linux 系统上一起运行，也会浪费掉宝贵的物理内存。如果将静态库换成动态库，那么这些问题都不会出现。动态库在发挥作用的过程中，必须经过两个步骤：

a) 链接编辑器(link editor)拿它和其他可重定位文件(.o 文件)以及其他可被共享的对象文件(shared object file)作为输入，经链接处理后，生成另外的可被共享的对象文件(shared object file)或者可执行的对象文件(Executable file)。

b) 在运行时，动态链接器(dynamic linker)拿它和一个可执行的对象文件 Executable file 以及另外一些 Shared object file 来一起处理，并在系统中运行。本文实现的动态加载就是采用的这种方式。

1.3.1 ELF 文件格式

ELF 文件格式提供了两种视图，分别是链接视图和执行视图，这与前一节讲的是一一对应的。所谓的两种视图的意思就是，一个 ELF 文件包含了两种信息，一种用于辅助编译器链接，另一种用于单片机程序的执行。

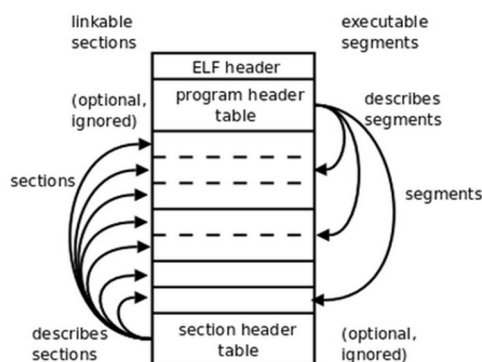


图 1.4 elf 文件的链接视图与执行视图

链接视图是以节（section）为单位，执行视图是以段（segment）为单位。链接视图就是在链接时用到的视图，而执行视图则是在执行时用到的视图。上图左侧的视角是从链接来看的，右侧的视角是执行来看的。整个文件可以分为四个部分：

- ELF 文件头：描述整个文件的组织。
- 程序头部表：描述文件中的各种 segments，用来告诉单片机如何组织程序。
- 节区或者段区：segments 是从运行的角度来描述 elf 文件，sections 是从链接的角度来描述 elf 文件，也就是说，在链接阶段，我们可以忽略 program header table 来处理此文件，在运行阶段可以忽略 section header table 来处理此程序（所以很多加固手段删除了 section header table）。从图中我们也可以看出，segments 与 sections 是包含的关系，一个 segment 包含若干个 section。
- 节区头部表：包含了文件各个 segction 的属性信息。

链接视图	执行视图
ELF 头部	ELF 头部
程序头部表（可选）	程序头部表
节区 1	段 1
...	
节区 n	段 2
...	
...	...
节区头部表	节区头部表（可选）

程序头部表（Program Header Table），如果存在的话，告诉系统如何创建进程映像。

节区头部表（Section Header Table）包含了描述文件节区的信息，比如大小、偏移等。

节区或者段区 包含了各种信息，其中节区的信息主要用于给链接器使用，段区则用于单片机加载数据，每个段区和节区都有自己的名称。

关于 ELF 文件的其他知识可以自行百度学习，这里需要对 ELF 文件有大概的了解，方便后续的讲解。

第二部分 动态加载器实现思路与环境搭建

本章首先理清动态加载器的实现思路。

2.1 什么是动态加载？

所谓动态加载就是让一部分函数独立于地址地执行，也就是可以从 SD 卡等存储介质读取出来，并且可以让函数在内存中的任意物理地址都可以执行。为了实现这样操作，我们首先要理清一个函数编译完成后，所有可能与内存物理地址有关的 C 语言对象，我主要归结为以下三个：

1.函数中调用的全局变量。这个比较好理解，因为全局变量必定会有个内存地址，在编译完成后的机器码中，全局变量就会以一个地址的形式出现在代码中。

2.函数中的静态变量。这个与全局变量类似，同样是在编译完成后以地址的形式出现在机器码中。

3.其他函数。在调用其他函数时，有时同样需要考虑到函数的真实物理地址在哪里。注意，并不是在所有情况下，调用函数都会是真实的物理地址。举个例子，当一个 C 文件中，foo_a 函数调用了本文件的静态函数 foo_b 时：

```
1. static void foo_b(void){  
2.     //源代码  
3. }  
4.  
5. void foo_a(void){  
6.     foo_b();  
7. }
```

foo_a 函数编译后的指令可能为：

```
1. LDR R0,=foo_b;  
2. BL R0;
```

编译器与链接器可以对其进行优化，可以将其优化为跳转到当前地址的相对偏移地址。原本需要两个指令的程序，就可以降低为一个指令，有效地降低了代码密度。这里的叙述并不严谨，实际的代码优化可能更加复杂。

综上所述，实现动态加载的关键就是需要找到一种方法，使我们可以任意地修改

待加载函数编译后的机器码中，所有与内存地址相关的代码，也就是让被加载的函数中所有静态地、全局的变量变得可控，可以在线地进行修改。这个操作又可以叫做**内存地址的重定位**。在静态编译代码时，重定位的操作主要由链接器执行，而我们进行动态加载时，则需要由单片机做这一部分的工作。

至此，动态加载的实现主要问题已经可以大致地确定下来，就是使单片机可以对将要动态加载的函数进行内存地址的重定位，通过前面的学习，我们可以知道，elf 文件是一个很好的载体，elf 文件中有一类特殊的节区，可以专门用于实现重定位操作，这部分节区以一个表格的形式记录了所有与内存地址相关的代码位置，以及将内存地址写入机器码的方法。但是这样的节区在 MDK 正常编译输出的.axf 文件中是不存在的，因为这部分节区是为了让编译器可以将众多的.o 文件编译到一起，.o 文件的内存物理地址还未确定，因此需要有重定位的节区供给链接器调用，而 MDK 最后编译输出的.axf 文件都是编译链接完成的最终输出，物理地址已经最终确定下来了，因此重定位节区也就没有了存在的意义。所以最后的动态加载器的实现其实可以归结为两个步骤：

第一、如何用 MDK 生成具有重定位节区的.axf 文件？

第二、如何在单片机上实现对具有重定位节区的.axf 文件动态链接与运行？

（注：这里可能有的读者会产生疑问，为什么不考虑使用.o 文件作为动态加载实现的载体呢？这里主要有两点考虑：第一、.o 文件与.axf 文件就像半成品与成品的区别，实现.o 文件的动态加载单片机需要做的链接工作较大，效率不高。第二、.o 文件的符号表导出不方便管理，具体的原因后面会解释。）

下面我们逐步解决这两个问题。

2.2 fromelf 工具使用

Fromelf 是用于查看 MDK 生成的 elf 文件必备的工具，与 armcc、armclang 等编译器在同一目录，可以通过命令行操作输出 elf 文件的许多信息，通过 fromelf 可以检查编译输出的.axf 文件是否带有可重定位的节区。

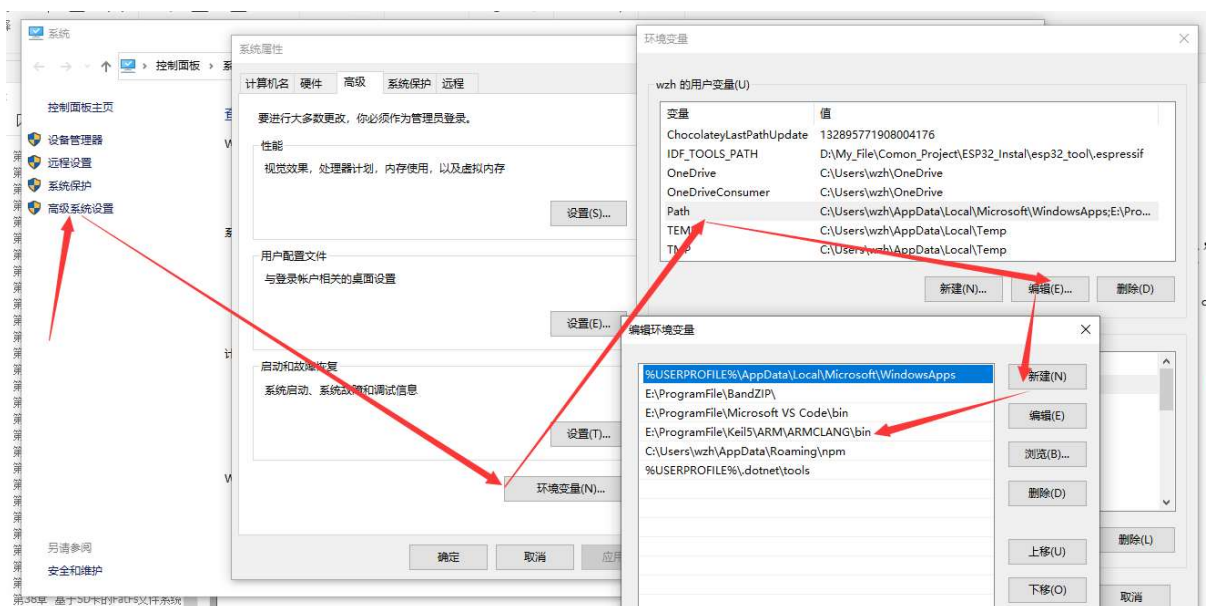
2.2.1 环境变量配置

调用此工具需要用到 Windows 的命令行工具，为了方便地使用 fromelf 工具，我们将其路径配置到环境变量中，路径在 Keil5 的安装目录下，我的编译器路径为：“E:\ProgramFile\Keil5\ARM\ARMCLANG\bin”。

下面以 win10 为例讲解环境变量配置步骤。

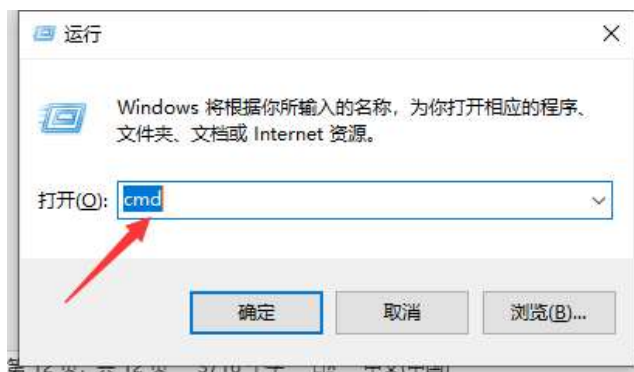


在文件管理器空白处，鼠标右键，选择属性。

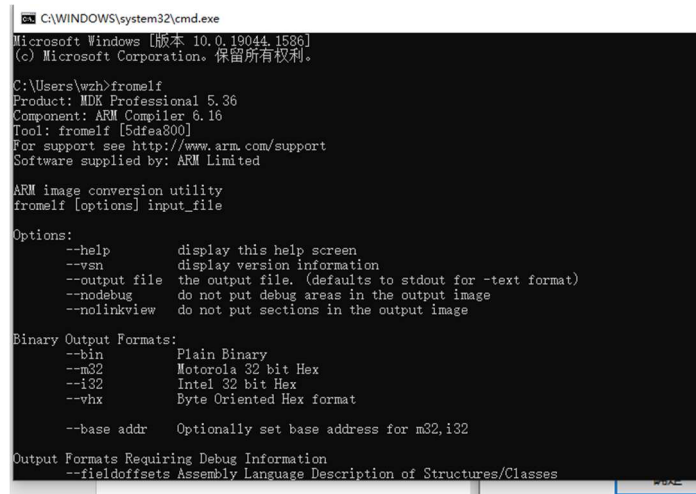


在弹出的页面中选择高级系统设置->环境变量->Path->编辑->新建。将 fromelf 工具的路径添加到环境变量中，随后依次点击确定。

随后使用组合键 Win+R，在弹出的窗口中输入 cmd 并回车，调用命令行窗口。



在命令行窗口中输入 `fromelf`，弹出以下信息证明环境变量添加成功。



```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [版本 10.0.19044.1586]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\wzh>fromelf
Product: MDK Professional 5.36
Component: ARM Compiler 6.16
Tool: fromelf [5dfea800]
For support see http://www.arm.com/support
Software supplied by: ARM Limited

ARM image conversion utility
fromelf [options] input_file

Options:
--help          display this help screen
--vsn           display version information
--output file   the output file. (defaults to stdout for -text format)
--nodebug       do not put debug areas in the output image
--nolinkview    do not put sections in the output image

Binary Output Formats:
--bin          Plain Binary
--m32          Motorola 32 bit Hex
--i32          Intel 32 bit Hex
--vbx          Byte Oriented Hex format
--base addr    Optionally set base address for m32,i32

Output Formats Requiring Debug Information
--fieldoffsets Assembly Language Description of Structures/Classes
```

2.2.2 fromelf 工具简单示例

使用 Win+R，敲入 `cmd` 字符，按下回车进入命令行窗口。



输入 `fromelf`，直接按下回车，可以得到的命令帮助

命令格式为：

`fromelf [options] input_file`

`options` 为命令的操作选项，这里我们主要使用 `--text` 选项获取 `elf` 文件的信息，`--text` 选项下的参数主要含义如图 2.2 所示。

```

C:\Users\wzh>fromelf
Product: MDK Professional 5.36
Component: ARM Compiler 6.16
Tool: fromelf [5dfea800]
For support see http://www.arm.com/support
Software supplied by: ARM Limited

ARM image conversion utility
fromelf [options] input_file

Options:
--help          display this help screen
--vsn           display version information
--output file   the output file. (defaults to stdout for -text format)
--nodebug       do not put debug areas in the output image
--nolinkview    do not put sections in the output image

Binary Output Formats:
--bin           Plain Binary
--m32           Motorola 32 bit Hex
--i32           Intel 32 bit Hex
--vbx          Byte Oriented Hex format

--base addr    Optionally set base address for m32,i32

Output Formats Requiring Debug Information
--fieldoffsets Assembly Language Description of Structures/Classes
--expandarrays Arrays inside and outside structures are expanded

Other Output Formats:
--elf           ELF
--text          Text Information

Flags for Text Information
-v             verbose
-a            print data addresses (For images built with debug)
-c            disassemble code
-d            print contents of data section
-e            print exception tables
-g            print debug tables
-r            print relocation information
-s            print symbol table
-t            print string table
-y            print dynamic segment contents
-z            print code and data size information

```

图 2.1 fromelf 命令帮助

fromelf 选项	可查看的信息
-v	详细信息
-a	数据的地址
-c	反汇编代码
-d	data section 的内容
-e	异常表
-g	调试表
-r	重定位信息
-s	符号表
-t	字符串表
-y	动态段内容
-z	代码及数据的大小信息

图 2.2 fromelf 的--text 子命令含义

下面我们使用 fromelf 工具查看一个普通的.axf 文件，示例的工程代码如下：

```

1.  #include <stdio.h>
2.
3.  int main(void) {
4.      int sum=0;
5.      for(int i=1;i<100;i++){
6.          sum+=i;
7.      }
8.      printf("1~100 的和为:%d\r\n",sum);
9.      return 0;
10. }

```

命令行输入：fromelf --text simulation.axf 查看相应的 axf 文件信息。由于输出的信息有点多，下面仅截取部分输出截图。

```

D:\My_File\Comon_Project\ARM_CortexM7Simulation\Project\Objects>fromelf --text simulation.axf
=====
** ELF Header Information
   File Name: simulation.axf
   Machine class: ELFCLASS32 (32-bit)
   Data encoding: ELFDATA2LSB (Little endian)
   Header version: EV_CURRENT (Current version)
   Operating System ABI: none
   ABI Version: 0
   File Type: ET_EXEC (Executable) (2)
   Machine: EM_ARM (ARM)
   Image Entry point: 0x000003c1
   Flags: EF_ARM_HASENTRY + EF_ARM_ABI_FLOAT_SOFT (0x05000202)
   ARM ELF revision: 5 (ABI version 2)
   Conforms to Soft float procedure-call standard

   Built with
   Component: ARM Compiler 6.16 Tool: armasm [5dfea300]
   Component: ARM Compiler 6.16 Tool: armlink [5dfeaa00]

   Header size: 52 bytes (0x34)
   Program header entry size: 32 bytes (0x20)
   Section header entry size: 40 bytes (0x28)

   Program header entries: 1
   Section header entries: 15

   Program header offset: 24596 (0x00006014)
   Section header offset: 24628 (0x00006034)

   Section header string table index: 14
=====
** Program header #0 (PT_LOAD) [PF_X + PF_W + PF_R + PF_ARM_ENTRY]
   Size : 8984 bytes (4536 bytes in file)
   Virtual address: 0x00000000 (Alignment 8)
=====
** Section #1 'ER_IROM1' (SHT_PROGBITS) [SHF_ALLOC + SHF_EXECINSTR]
   Size : 4516 bytes (alignment 8)
   Address: 0x00000000
=====
** Section #2 'RW_IRAM1' (SHT_PROGBITS) [SHF_ALLOC + SHF_WRITE]
   Size : 16 bytes (alignment 8)
   Address: 0x20000000
=====

```

观察节区属性可以看到，没有用于重定位的节区，可以重定位的节区的节区属性为 SHT_REL 类型。

```
** Section #6 '.dyn' (SHT_REL)
   Size   : 48 bytes (alignment 4)
   Symbol table #4 '.dynsym'
   6 relocations applied to section #0 '[Anonymous Section]'
```

图 2.3 可重定位的节区示例

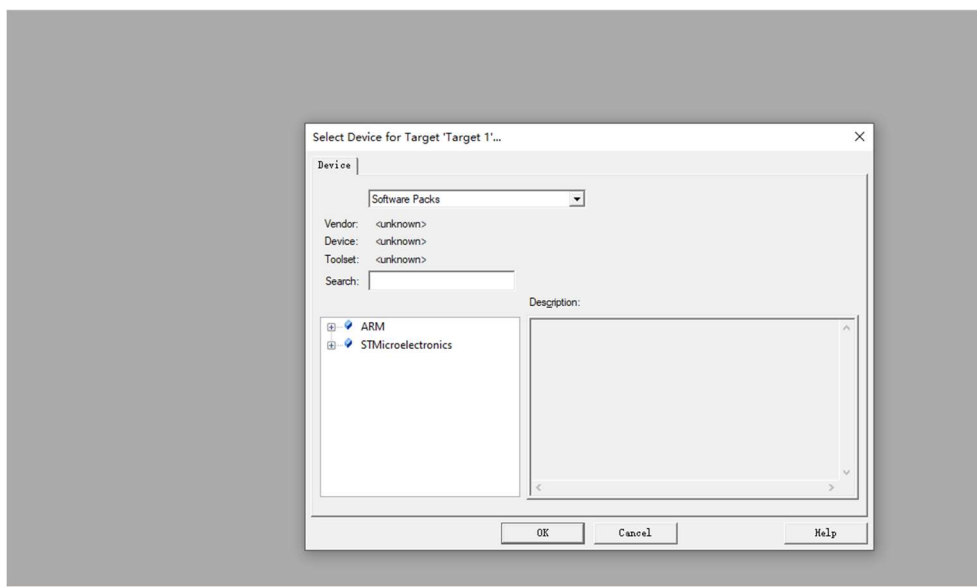
那么，如何生成具有可重定位的节区的.axf 文件呢？关键在两点，一是链接器的命令行配置，二是链接器文件的编写，下一章着重讲解一下这种工程的搭建方法。

第三部分 生成具有重定位节区的.axf 文件

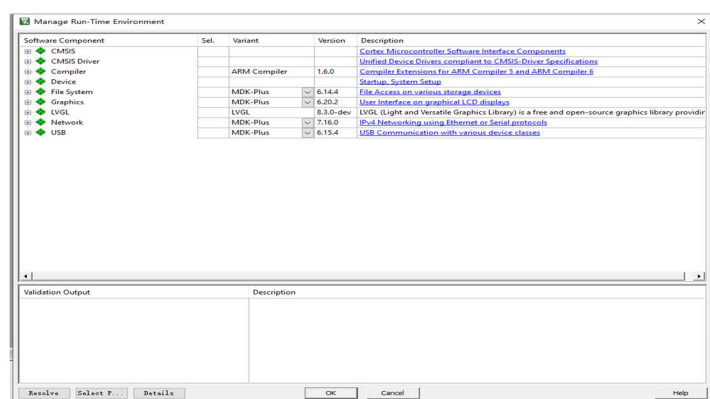
3.1 输出具有重定位节区的.axf 工程搭建

1.新建一个工程。

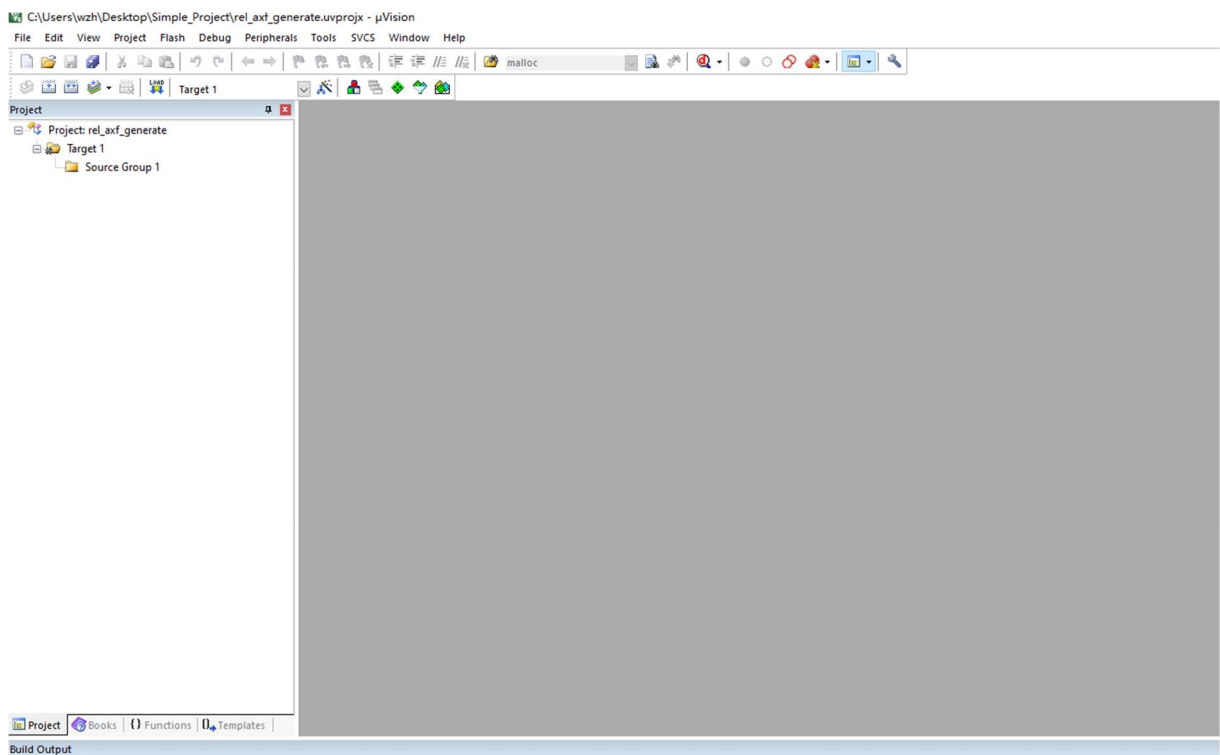
2.在下面的对话框选择与你的单片机芯片架构一致的单片机型号，比如我使用的是 STM32H743 这款芯片，那么我既可以在 ST 芯片目录下选择这款芯片，也可以选择其他的 Cortex-M7 双精度浮点的芯片，最重要的是内核要一致，保证编译出来的指令是可以使用的。



3.RTE 环境不需要配置



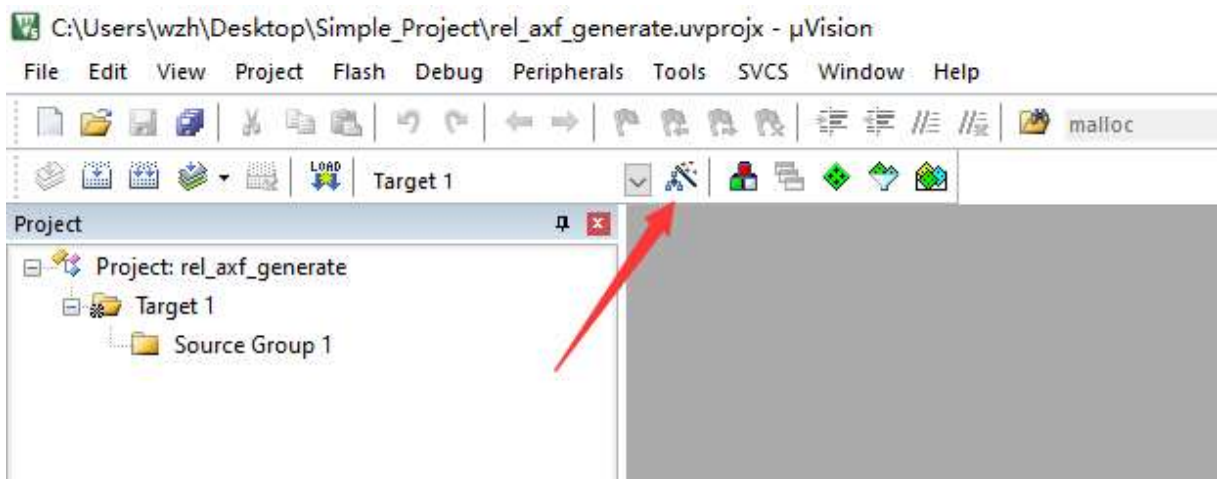
4.建好的工程长这个样子。



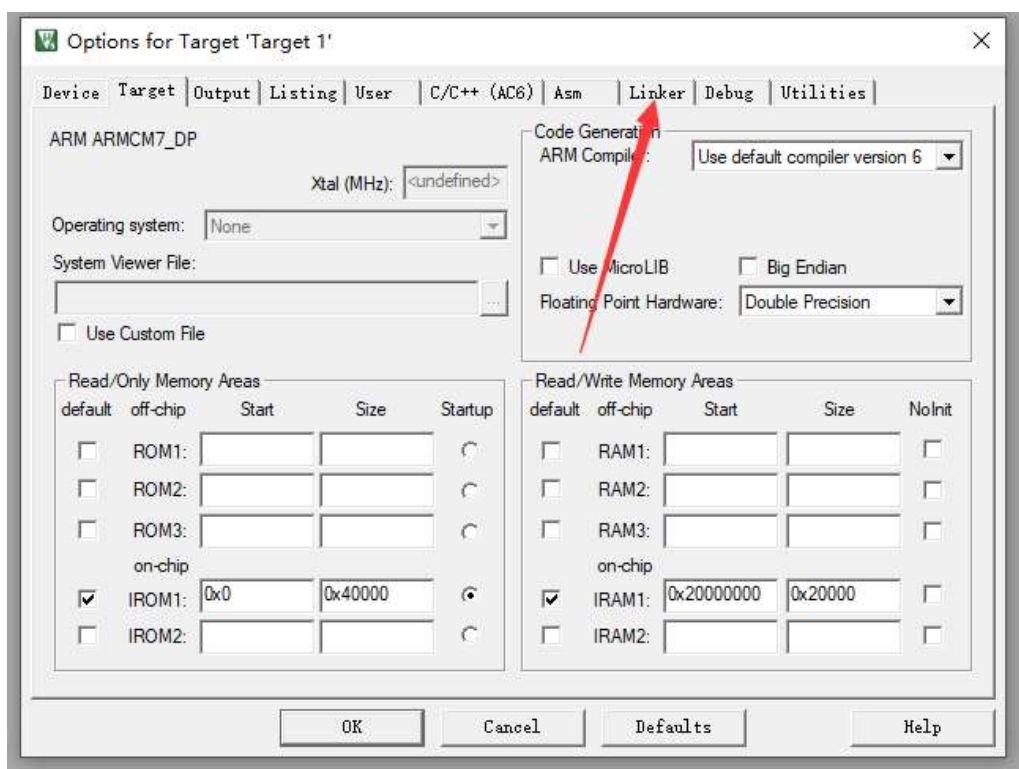
5.接下来是重点。首先在工程目录下，新建一个名为 `dll_generate.sct` 的分散加载文件。

Listings	2022/4/15 17:17	文件夹	
Objects	2022/4/15 17:17	文件夹	
rel_axf_generate.uvoptx	2022/4/15 17:16	UVOPTX 文件	5 KB
rel_axf_generate.uvprojx	2022/4/15 17:16	uVision5 Project	16 KB
dll_generate.sct	2022/4/15 17:21	Windows Script ...	0 KB

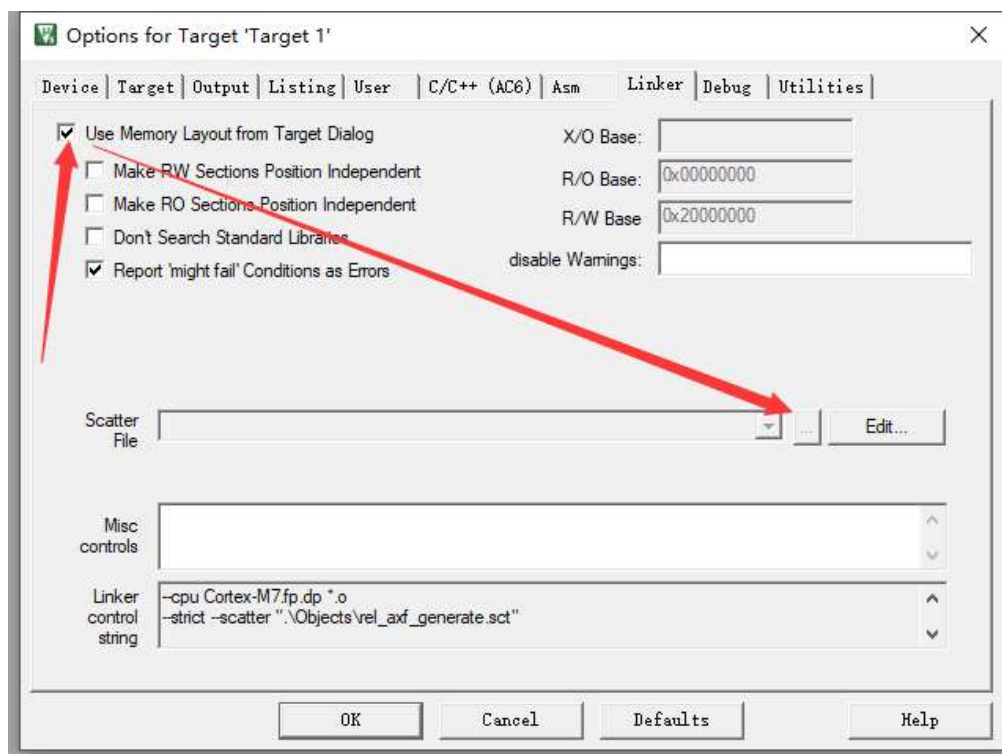
6.回到 MDK 的配置界面，点击小魔术棒进入工程配置选项。

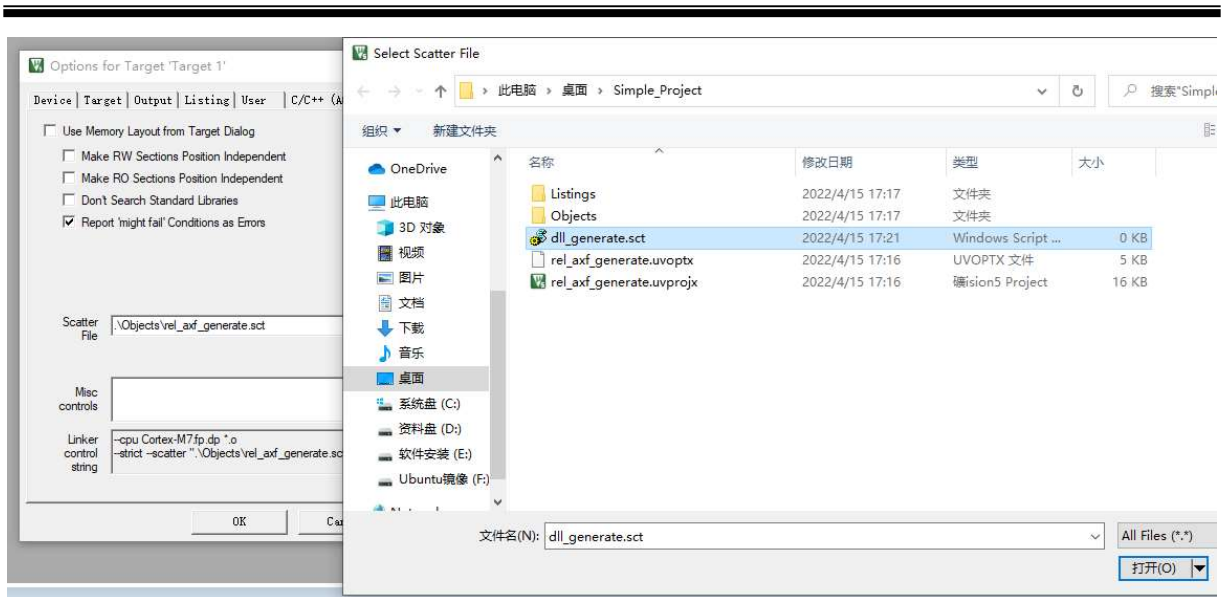


7. 点击工程选项卡的 Link 选项，进入链接器配置。

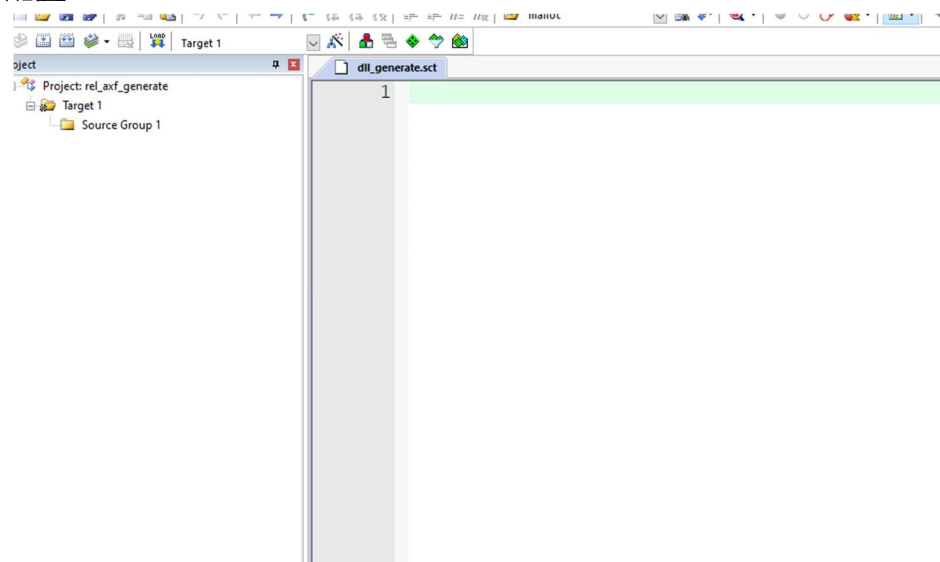


8. 依次点击 Use Memory Layout from Target Dialog->..., 在弹出来的文件选择对话框中选择我们新建的.sct 文件。

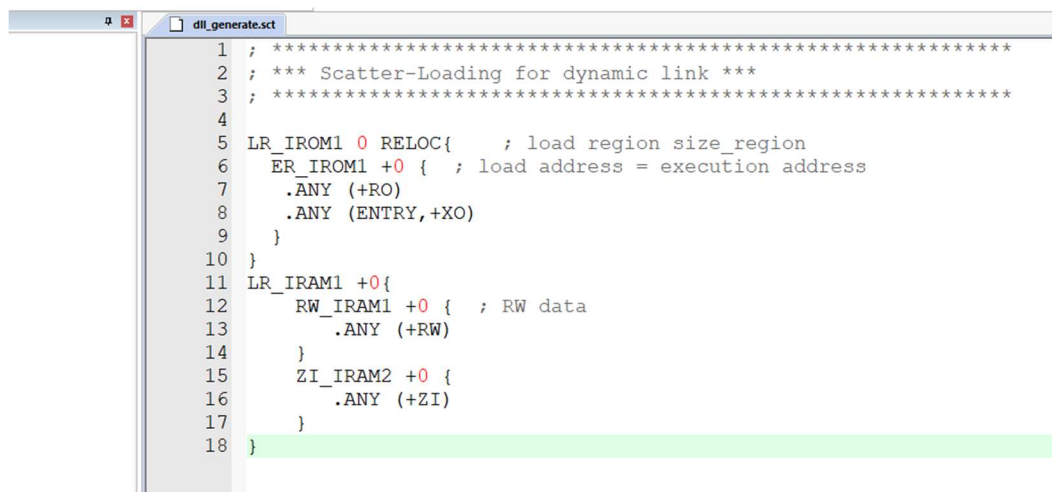




9. 点击确定，随后点击 Edit 对 `dll_generate.sct` 文件进行编辑，然后点击 Ok 先暂时退出工程配置。

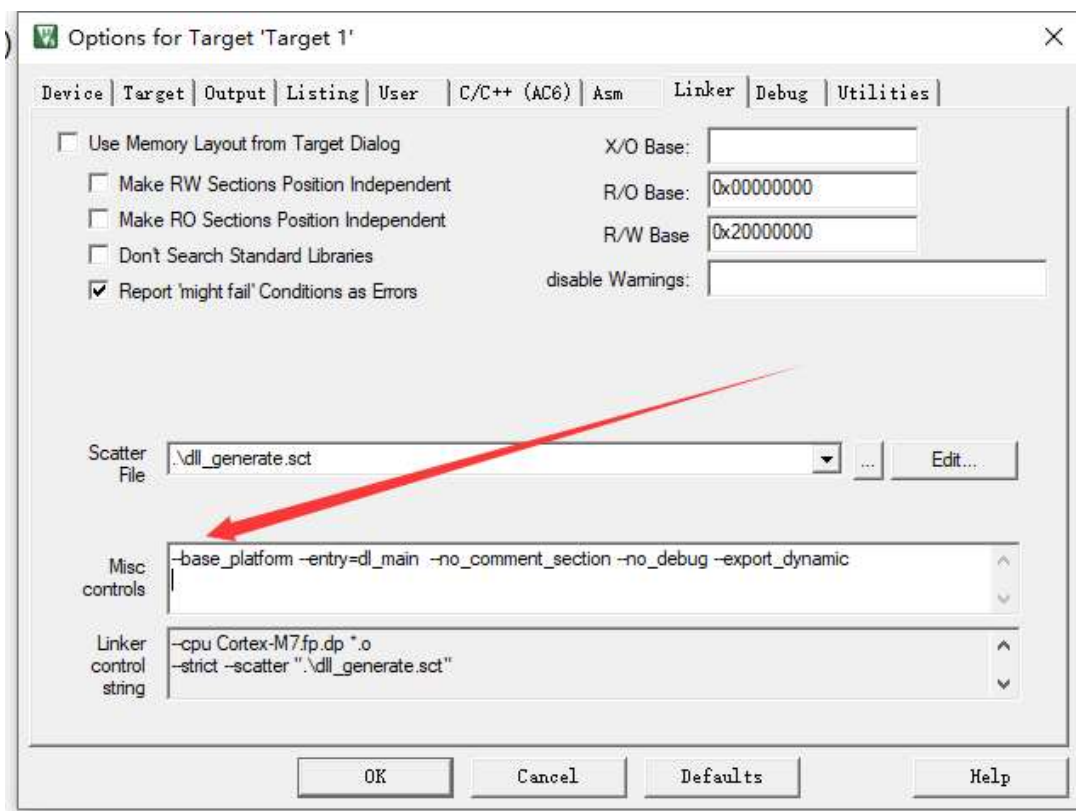


10. 在该 sct 文件中，输入以下内容。



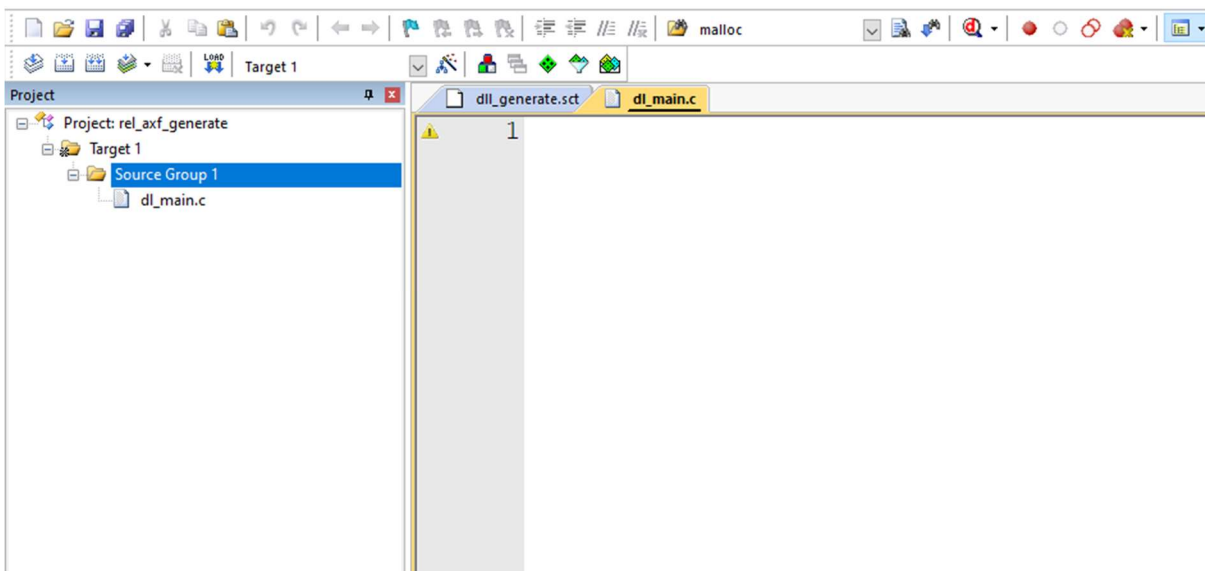
11.随后，回到链接器配置选项卡，并在 Misc controls 栏下输入如下命令行参数。

--base_platform --entry=dl_main --no_comment_section --no_debug --export_dynamic

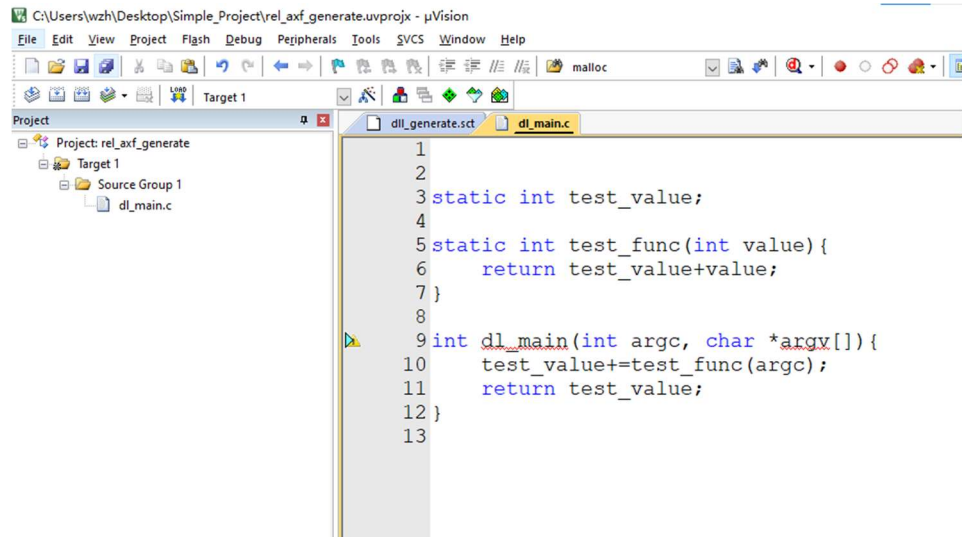


12.点击 Ok，回到原来的工程中。

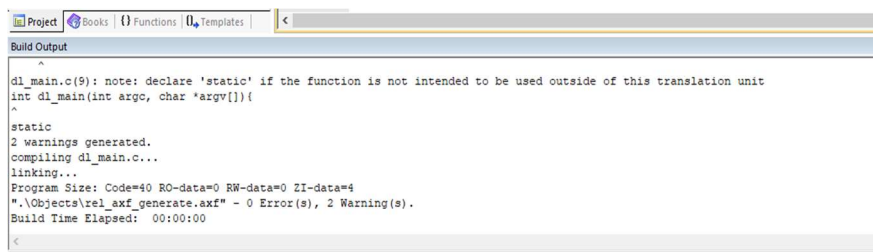
13.新建一个 c 语言源文件，并将其添加到工程中，起名为 dl_main.c



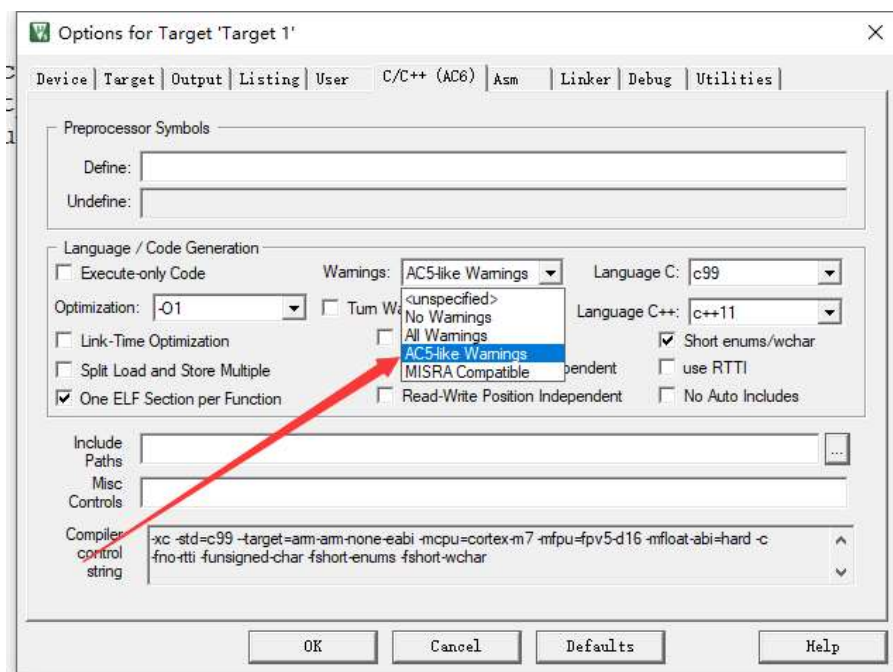
14.在源文件中，添加如下的代码，并编译。



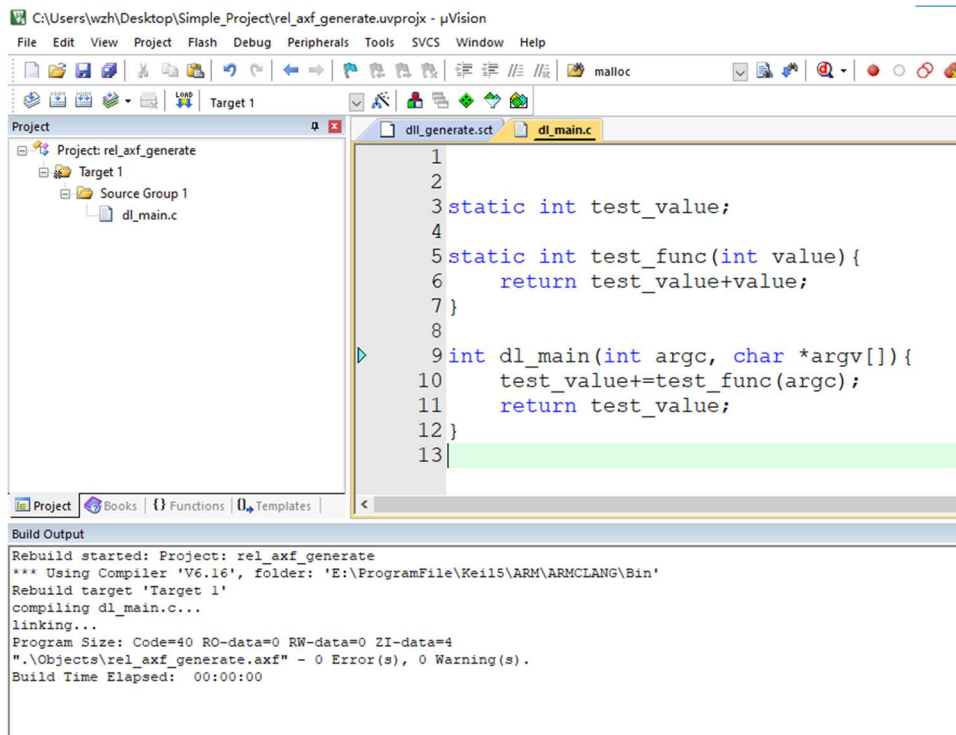
15.如果一切顺利的话，应该会报几个警告。



16.主要原因是我的工程默认是 V6 编译器，警告全开，因此有点敏感，重新打开小魔术棒，点击 C/C++选项，把警告改为 AC5 Like Warning。



17.更改完后，一般编译都不会出现警告了，如果出现编译错误，请从步骤 1 开始重新检查操作步骤有没有错误。



18.还是运行 fromelf 工具,检查以下此时生成的 axf 文件是否具有可重定向的节区。
在命令行窗口输入 `fromelf -text -r rel_axf_generate.axf`

```
C:\Users\wzh\Desktop\Simple_Project\Objects>fromelf --text -r rel_axf_generate.axf
=====
** ELF Header Information

File Name: rel_axf_generate.axf

Machine class: ELFCLASS32 (32-bit)
Data encoding: ELFDATA2LSB (Little endian)
Header version: EV_CURRENT (Current version)
Operating System ABI: ARM BPABI with Symbol Versioning
ABI Version: 0
File Type: ET_EXEC (Executable) (2)
Machine: EM_ARM (ARM)

Image Entry point: 0x00000001
Flags: EF_ARM_HASENTRY + EF_ARM_ABI_FLOAT_HARD (0x05000402)

ARM ELF revision: 5 (ABI version 2)

Conforms to Hard float procedure-call standard

Header size: 52 bytes (0x34)
Program header entry size: 32 bytes (0x20)
Section header entry size: 40 bytes (0x28)

Program header entries: 3
Section header entries: 12

Program header offset: 52 (0x00000034)
Section header offset: 644 (0x00000284)

Section header string table index: 11
=====
** Program header #0 (PT_LOAD) [PF_X + PF_R + PF_ARM_ENTRY]
Size : 40 bytes
Virtual address: 0x00000000 (Alignment 8)
=====
```

可以在节区表中,找到这样的一个具有重定位节区,下面信息就是重定位表,包含了重定位符号的偏移位置以及需要进行的重定位操作。


```

** Section #6 '.dyn' (SHT_REL)
   Size      : 32 bytes (alignment 4)
   Symbol table #4 '.dynsym'
   4 relocations applied to section #0 '[Anonymous Section]'

#      Offset      Relocation Type      Wrt Symbol      Defined in
=====
0  0x00000006    47 R_ARM_THM_MOVW_ABS_NC    1 test_value    #2 'ZI_IRAM2'
1  0x0000000a    48 R_ARM_THM_MOVT_ABS      1 test_value    #2 'ZI_IRAM2'
2  0x00000018    47 R_ARM_THM_MOVW_ABS_NC    1 test_value    #2 'ZI_IRAM2'
3  0x0000001c    48 R_ARM_THM_MOVT_ABS      1 test_value    #2 'ZI_IRAM2'

```

至此，输出具有重定位节区的.axf 文件模板工程就搭建完毕了，是不是看起来很简单？比正常的单片机裸机工程还要简洁，连启动文件都不需要，仅仅一个函数源文件即可使用。下面着重讲解以下搭建这个工程的原理，命令行配置的含义，sct 文件的含义，以及如何这个工程生成的.axf 文件如何使用。

3.2 工程搭建的原理

在工程编译完成后，链接器就会开始工作，将代码中包含地址的部分链接到真实存在的物理地址中。编译器输出的.o 文件本来就是带重定向信息的，不需要我们过多的干涉，因此实现的重点就在编译器的配置上。

在《Arm® Compiler Version 6.16 Reference Guide》这本手册的 Part-C 部分，详细地介绍了关于 armlink 的特性，在这一部分的 C2 篇章中(C2-537 页)，介绍了 armlink 链接器所支持的所有链接模型，一共有五种。

1.裸机

这种是我们最常用的，也就是我们平时使用的单片机工程都在此列，可以选择使用.sct 文件进行分散加载，所有的代码和变量都会被分配到相应的物理地址上。

2.裸机生成位置独立的可执行文件

这种根据手册中的描述，是可以生成位置独立的代码的，个人猜测具体的操作大致应该是将整个代码看成一个整体，然后在某个固定的寄存器存放整个代码的偏移地址，编译时所有与地址有关的代码都变成该寄存器+偏移地址的方式。此方式在手册上标记为弃用，没有做深入的研究。

3.部分链接模型

这个链接模型就是可以把零散的编译输出.o 文件，合并成一个大的.o 文件，这种方式其实也是可以用于实现动态加载，但考虑效率方面的问题，.o 文件需要单片机进行较多的链接操作，因此最后并未使用此方案。

4.基础平台二进制接口（BPABI）链接模型

这种模型主要用于 Linux 等平台实现动态加载，STM32 不支持。

5.基础平台链接模型

在基础平台二进制接口（BPABI）链接模型中，有一些与内存映射有关的限制，我的理解是这些限制可能规定了被加载的 elf 文件与系统之间 API 是如何调用的，在这个链接模型中没有这些限制了，因此可以使得链接器不限制目标平台地支持动态链接，自然支持 STM32 了。手册后面还讲了 SysV 链接模型等，不太清楚这个是由于什么场景，这部分未作研究，有兴趣的可以看看。

了解完这些后，我们在回过头来看，为什么链接器配置的命令行是这样的。此部分同样可以结合《Arm® Compiler Version 6.16 Reference Guide》这本手册的 Part-C 部分中的 C1 部分进行理解。

`--base_platform`

此命令行是用于开启链接器的基础平台链接模型，让生成的.axf 具有动态信息，也就是具有重定位节区。

`--entry=dl_main`

此命令行是用于将整个程序的入口地址设置为 dl_main 函数，在 arm 的基础平台链接模型中，最后输出是一个.axf 文件，我们使用 fromelf 工具进行文件信息查看时就会发现，生成的.axf 文件依旧是可执行的 elf 文件类型，既然是可执行的文件，那么就会存在一个程序入口地址，所谓程序的入口地址就类似于裸机中的 main 函数（说是__main 函数更为准确）。但是为什么不用原来的 main 函数呢？主要原因为 main 函数是隶属于 C 库函数的，在调用 main 函数前，C 库内部会进行初始化，设置堆指针等操作，而我们这个工程连启动文件都没有，堆区和初始化自然无从谈起了，为了规避这个问题，我们索性重新定义个新的入口。

```
C:\Users\wzh\Desktop\Simple_Project\Objects>fromelf --text -r rel_axf_generate.axf
=====
** ELF Header Information
   File Name: rel_axf_generate.axf
   Machine class: ELFCLASS32 (32-bit)
   Data encoding: ELFDATA2LSB (Little endian)
   Header version: EV_CURRENT (Current version)
   Operating System ABI: ARM BPABI with Symbol Versioning
   ABI Version: 0
   File Type: ET_EXEC (Executable) (2)
   Machine: EM_ARM (ARM)

   Image Entry point: 0x00000001
   Flags: EF_ARM_HASENTRY + EF_ARM_ABI_FLOAT_HARD (0x05000402)

   ARM ELF revision: 5 (ABI version 2)
   Conforms to Hard float procedure-call standard
```

程序入口

```
--no_comment_section --no_debug
```

这两个命令比较简单，就是为了缩小.axf文件的体积，移除了注释段和调试信息。

```
--export_dynamic
```

这个命令是用于导出动态符号的。开启了这个选项，.axf文件就会多出动态符号节区，动态符号节区是由一对对字符串和偏移地址组成的，假如外部的程序想要调用我们输出的.axf文件内部的某一个函数，那么它就可以根据动态符号表查找到相应的函数在哪个地址，提升了程序的灵活性。默认这个特性是关闭的，也就是说我们原本想要调用.axf文件函数的方式只能通过获取程序入口点，然后通过dl_main函数进行间接调用，通过开启这个选项就可以打破这个限制。那么，什么符号会被导出到符号表里呢？对于这种情况，在AC6编译器中有专门的指令，指令如下：

```
1.  #define DL_EXPORT      __attribute__((visibility("default")))
2.  #define DL_HIDDEN      __attribute__((visibility("hidden")))
```

在我们需要导出到符号表的函数前面或者后面，添加这个后缀__attribute__((visibility("default"))修饰一下，编译器就会将此函数的函数名与相对地址自动导出到符号表中，而在.o文件实现的动态加载中无法方便的管理导出的函数。推荐大家只用它修饰函数，虽然也可以修饰变量，但是我在后面的动态加载器的实现中出于性能考虑将除去函数以外的所有符号都过滤掉了，因此变量类型的动态符号将不会被识别到。

3.3 分散加载文件编写依据

其实这个分散加载文件编写的比较随意，远没有链接器命令行设置部分那样考究，原有的分散加载文件是带有绝对地址信息的，每一段大小是固定的。而我这个分散加载文件每个段大小都不是固定的，起始地址设置为0，并且也不限制范围地分配内存，判断内存是否足够的任务交由动态加载器去判断，关于分散加载文件的语法同样可以参照《Arm® Compiler Version 6.16 Reference Guide》的PartC-C7部分进行学习。

3.4 小结

至此，关于动态加载器实现的第一个主要部分，《具有重定位信息的.axf文件生成》就告一段落了，下章讲解动态加载器的实现。本人水平有限，如果有哪一部分讲解的不够准确或者讲的不够清楚的，欢迎大家在论坛帖子下告知，一起交流学习。🥰

第四部分 动态加载器的实现

在上一章中，我们实现了具有重定位信息的.axf 文件的生成，本章则开始讲解如何读取利用上一章生成的.axf 文件，本章知识与 elf 文件格式联系较为紧密，想要深入了解的话可以看我帖子下面的《elf 文件格式中文版》进行学习。

4.1 动态加载器 API 介绍

如果从 elf 文件格式出发，一步一步地去实现.axf 文件的加载器是比较复杂的，假如有一个经过测试可以正常使用的动态加载器代码可以参照的话，对于我们的移植工作是有利的，我参考的是国产操作系统 RT-Thread 实现的动态加载。

其中有关动态加载部分的在 RT-Thread 源码中的这个文件夹下：

> 此电脑 > 资料盘 (D:) > My_File > 资料 > STM32资料 > rt-thread-gitee_master > components > libc > libdl				
名称	修改日期	类型	大小	
arch	2021/11/23 13:23	文件夹		
dlclose.c	2021/10/15 16:17	C 源文件	1 KB	
dlelf.c	2021/10/15 16:17	C 源文件	17 KB	
dlelf.h	2021/10/15 16:17	C Header 源文件	18 KB	
dLError.c	2021/10/15 16:17	C 源文件	1 KB	
dlfcn.h	2021/10/15 16:17	C Header 源文件	1 KB	
dlmodule.c	2021/10/15 16:17	C 源文件	23 KB	
dlmodule.h	2021/10/15 16:17	C Header 源文件	3 KB	
dlopen.c	2021/10/15 16:17	C 源文件	2 KB	
dlsym.c	2021/10/15 16:17	C 源文件	1 KB	
SConscript	2021/10/15 16:17	文件	1 KB	

其中最为关键的是 dlelf.c 这个文件，这个文件内有两个函数分别实现了编译好的 elf 文件的动态加载（也就是.axf 文件）与.o 文件的动态加载。

实现编译好的 elf 文件动态加载的函数名：

```
rt_err_t dlmodule_load_shared_object(struct rt_dlmodule* module, void *module_ptr)
```

实现.o 文件动态加载的函数名：

```
rt_err_t dlmodule_load_relocated_object(struct rt_dlmodule* module, void *module_ptr)
```

rt-thread 的 elf 文件加载器的实现有一部分与系统本身耦合严重，并且还有一部分并不适合在 MDK 环境下使用，我对其进行了一定的裁剪，仅保留了必要的功能。

在讲解源码之前，我先从使用者的角度，对整个动态加载器的 API 进行说明。根据

我们日常的使用习惯，动态加载器的 API 函数可以精简为以下四个：

一、加载.axf 文件到句柄

```
1. //加载 elf 文件到句柄中
2. //pHandle: 句柄
3. //lib_ptr: 文件缓存首地址
4. //返回值: DL_Err_Type 解析过程中遇到的错误
5. DL_Err_Type dl_load_lib(DL_Handler* pHandle,const void* const lib_ptr)
```

这个函数的功能就是将已经读取到内存中的.axf 文件进行解析，将.axf 文件恢复为单片机可以执行的程序镜像，并将程序镜像中的 main 函数地址、需要动态导出的函数地址与函数名等，保存在 pHandle 中，方便我们后续操作。

二、获取.axf 文件中某一个函数的地址

```
1. //获取库中的函数指针
2. //pHandle: 句柄
3. //sys_name: 函数名
4. //返回值: void* 函数指针（等于 NULL 时，表示未找到该函数）
5. void* dl_get_func(DL_Handler* pHandle,const char* sys_name)
```

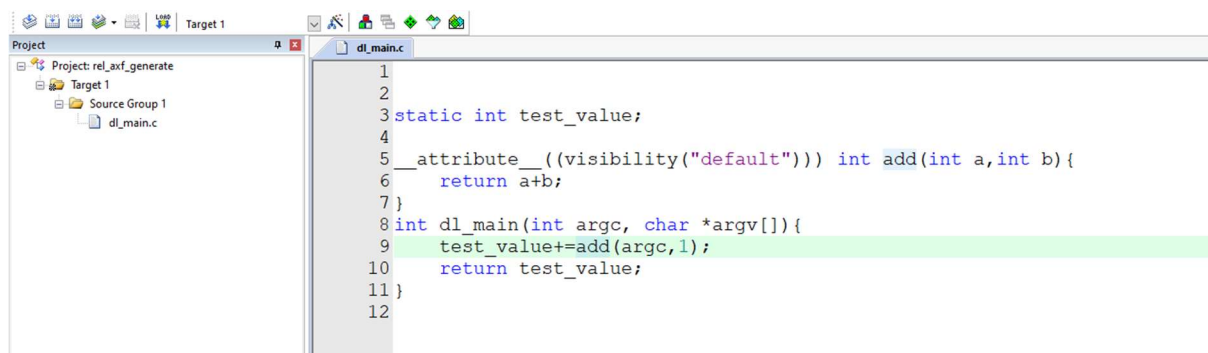
这个函数就是用来寻找，我们编译 .axf 文件的源码工程中，所有用 `__attribute__((visibility("default")))`

后缀修饰过的函数，比如，在我们编译可动态加载的.axf 文件的工程源码中，有着这样的一段代码：

```
1. __attribute__((visibility("default"))) int add(int a,int b){
2.     return a+b;
3. }
```

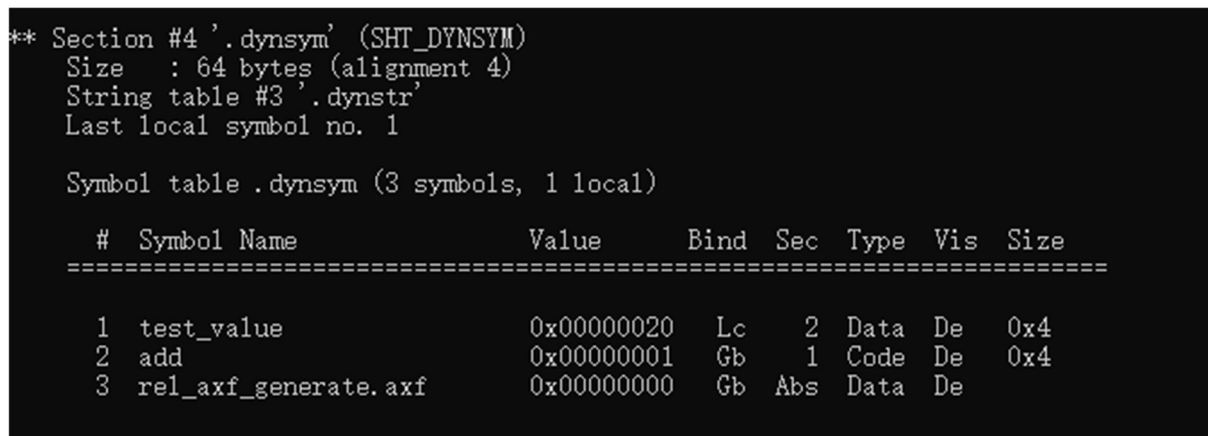
这里的 add 函数就会被导出到动态符号表中，可以通过 fromelf 工具查看我们当前所有被导出到动态符号表中的函数符号：

测试工程的代码如下：



```
1
2
3 static int test_value;
4
5 __attribute__((visibility("default"))) int add(int a,int b){
6     return a+b;
7 }
8 int dl_main(int argc, char *argv[]){
9     test_value+=add(argc,1);
10    return test_value;
11 }
12
```

使用的 fromelf rel_axf_generate.axf --text -s 命令的查看当前所有的符号：



```
*** Section #4 '.dynsym' (SHT_DYNSYM)
Size : 64 bytes (alignment 4)
String table #3 '.dynstr'
Last local symbol no. 1

Symbol table .dynsym (3 symbols, 1 local)

# Symbol Name Value Bind Sec Type Vis Size
=====
1 test_value 0x00000020 Lc 2 Data De 0x4
2 add 0x00000001 Gb 1 Code De 0x4
3 rel_axf_generate.axf 0x00000000 Gb Abs Data De
```

在.dynsym 节区中，就可以看到 add 函数的身影，.dynsym 节区就是专门用于管理所有被导出的动态符号使用的。

此时假设这个 .axf 文件已经被加载到了句柄 pHandle 中，那么调用 dl_get_func(pHandle,"add")就会返回 add 函数的函数指针。

三、获取.axf 文件的 main 函数地址

1. //获取库中的函数指针
2. //pHandle: 句柄
3. //返回值: (int*)(int, char *[]) 函数指针（等于 NULL 时，表示没有入口函数）
4. int(*dl_get_entry(DL_Handler* pHandle))(int argc, char *argv[])

这个函数就是用于获取.axf 文件中 main 函数的地址的，功能比较简单。

四、回收句柄

1. //释放加载库的内存
2. //pHandle: 句柄
3. void dl_destroy_lib(DL_Handler* pHandle)

因为我们在构建程序镜像的时候，必然要涉及到使用类似 `malloc` 函数分配程序所需要的静态内存、代码等，这个函数就是用于释放在这个过程中产生的内存分配的。

上面四个函数就是我们的动态加载器所需要实现的全部功能，是不是比较清晰呢？下面介绍下每个函数的实现细节，这部分内容比较枯燥，感兴趣的可以看看。动态加载器的源码我放在了 gitee 上：https://gitee.com/wzh1845462801/dynamic_loader

4.2 动态加载器 API 实现细节

整个动态加载器的核心就是 `dl_load_lib` 函数，因此仅着重讲解下此函数的实现。

```
1. //加载 elf 文件到句柄中
2. //pHandle: 句柄
3. //lib_ptr: 文件缓存首地址
4. //返回值: DL_Err_Type 解析过程中遇到的错误
5. DL_Err_Type dl_load_lib(DL_Handler* pHandle, const void* const lib_ptr){
6.     bool has_vstart=false;
7.     Elf_Addr vstart_addr=0, vend_addr=0;
8.     uint32_t dynsym_off;
9.     uint32_t max_align;
10.    DL_Err_Type err=DL_NO_ERR;
11.
12.
13.    if(memcmp(elf_header(lib_ptr)->e_ident,ELFMAG,SELFMAg)!=0){
14.        err=DL_FILE_INVALID;
15.        goto fail0;
16.    }
17.
18.    if(elf_header(lib_ptr)->e_ident[EI_CLASS]!=ELFCLASS32){
19.        err=DL_PLATFORM_ERR;
20.        goto fail0;
21.    }
22.
23.    if(elf_header(lib_ptr)->e_type!=ET_DYN&&elf_header(lib_ptr)->e_type!=ET_EXEC){
24.        err=DL_ELF_NOT_SUPPRORT;
25.        goto fail0;
26.    }
27.
28.
29.    //计算加载区的首地址以及所有加载区的大小
30.    for(size_t i = 0;i<elf_header(lib_ptr)->e_phnum;i++){
31.        if(elf_phdr(lib_ptr)[i].p_type!=PT_LOAD)
32.            continue;
```

```

33.         DL_LOG_D("LOAD segment: %d, 0x%p, 0x%08x\r\n", i, (void*)elf_phdr(lib_ptr)[i]
        ].p_vaddr, elf_phdr(lib_ptr)[i].p_memsz);
34.         if (elf_phdr(lib_ptr)[i].p_memsz < elf_phdr(lib_ptr)[i].p_filesz){
35.             DL_LOG_E("invalid elf: segment %d: p_memsz: %d, p_filesz: %d",\
36.                 i, elf_phdr(lib_ptr)[i].p_memsz, elf_phdr(lib_ptr)[i].p_filesz);
37.             err=DL_FILE_INVALID;
38.             goto fail0;
39.         }
40.         if(has_vstart==false){
41.             vstart_addr = elf_phdr(lib_ptr)[i].p_vaddr;
42.             vend_addr = elf_phdr(lib_ptr)[i].p_vaddr + elf_phdr(lib_ptr)[i].p_memsz;

43.             has_vstart = true;
44.             max_align = elf_phdr(lib_ptr)[i].p_align;
45.             if (vend_addr < vstart_addr){
46.                 DL_LOG_E("invalid elf: segment %d: p_vaddr: %d, p_memsz: %d",
47.                     i, elf_phdr(lib_ptr)[i].p_vaddr, elf_phdr(lib_ptr)[i].p_m
48.                     emsz);
49.                 err=DL_FILE_INVALID;
50.                 goto fail0;
51.             }
52.             else{
53.                 if (elf_phdr(lib_ptr)[i].p_vaddr < vend_addr){
54.                     DL_LOG_E("invalid elf: segment should be sorted and not overlapped")
55.                     ;
56.                     err=DL_FILE_INVALID;
57.                     goto fail0;
58.                 }
59.                 vend_addr = elf_phdr(lib_ptr)[i].p_vaddr + elf_phdr(lib_ptr)[i].p_memsz;

60.                 if (vend_addr < elf_phdr(lib_ptr)[i].p_vaddr){
61.                     DL_LOG_E("invalid elf: "
62.                         "segment %d address overflow", i);
63.                     err=DL_FILE_INVALID;
64.                     goto fail0;
65.                 }
66.                 if(elf_phdr(lib_ptr)[i].p_align>max_align){
67.                     max_align=elf_phdr(lib_ptr)[i].p_align;
68.                 }
69.             }
70.         }

```



```
71.
72.    //根据加载区大小分配内存
73.    pHandle->mem_size=vend_addr-vstart_addr;
74.    pHandle->vstart_addr=vstart_addr;
75.    DL_LOG_D("module size: %d, vstart_addr: 0x%p", pHandle->mem_size, (void*)vstart_
    addr);
76.    if(pHandle->mem_size==0){
77.        DL_LOG_E("Module: size error\n");
78.        err=DL_MEMSIZE_ERR;
79.        goto fail0;
80.    }
81.    pHandle->mem_ptr=dl_port_aligned_alloc(max_align,pHandle->mem_size);
82.    if(pHandle->mem_ptr==NULL){
83.        DL_LOG_E("Module: allocate space failed.");
84.        err=DL_MALLOC_FAIL;
85.        goto fail0;
86.    }
87.    memset(pHandle->mem_ptr,0,pHandle->mem_size);
88.
89.    //复制文件数据到加载区上
90.    for(size_t i=0;i<elf_header(lib_ptr)->e_phnum;i++){
91.        if(elf_phdr(lib_ptr)[i].p_type==PT_LOAD){
92.            memcpy((uint8_t*)pHandle->mem_ptr+elf_phdr(lib_ptr)[i].p_vaddr-
            vstart_addr,\
93.                (uint8_t*)lib_ptr+elf_phdr(lib_ptr)[i].p_offset,\
94.                elf_phdr(lib_ptr)[i].p_filesz);
95.        }
96.    }
97.
98.    //此处区分共享库与可执行文件
99.    if(elf_header(lib_ptr)->e_entry!=0){//若存在程序入口点
100.        pHandle->entry_addr=(uint8_t*)pHandle->mem_ptr + elf_header(lib_ptr)->e_ent
        ry - vstart_addr;
101.    }
102.    else{
103.        pHandle->entry_addr=NULL;
104.    }
105.
106.    //开始重定向
107.    //寻找带有重定向信息的节区
108.    for(size_t i=0;i<elf_header(lib_ptr)->e_shnum;i++){
109.        Elf_Sym *symtab;
110.        Elf_Rel *rel;
```

```

111.     char *strtab;
112.     size_t nr_reloc;
113.     if (!IS_REL(elf_shdr(lib_ptr)[i]))
114.         continue;
115.
116.     //获取重定向表项的表头
117.     rel = (Elf_Rel *)((uint8_t *)lib_ptr + elf_shdr(lib_ptr)[i].sh_offset);
118.     //获取动态符号表的表头
119.     symtab = (Elf_Sym *)((uint8_t *)lib_ptr + elf_shdr(lib_ptr)[elf_shdr(lib_ptr)[i].sh_link].sh_offset);
120.     //获取动态符号代表的字符串表的表头
121.     strtab = (char *)((uint8_t *)lib_ptr + elf_shdr(lib_ptr)[elf_shdr(lib_ptr)[elf_shdr(lib_ptr)[i].sh_link].sh_link].sh_offset);
122.     nr_reloc = (size_t)(elf_shdr(lib_ptr)[i].sh_size / sizeof(Elf_Rel));
123.     for(size_t j=0;j<nr_reloc;j++){
124.         extern DL_Err_Type dl_relocate(DL_Handler *pHandle, Elf32_Rel *rel, Elf32_Addr sym_val);
125.         Elf_Sym *sym = &symtab[ELF32_R_SYM(rel[j].r_info)];
126.
127.         DL_LOG_D("relocate symbol %s shndx %d", strtab + sym->st_name, sym->st_shndx);
128.         if ((sym->st_shndx != SHT_NULL) || (ELF_ST_BIND(sym->st_info) == STB_LOCAL)) { //如果符号可以解析
129.             Elf_Addr addr;
130.
131.             addr = (Elf_Addr)((uint8_t *)pHandle->mem_ptr + sym->st_value - vstart_addr);
132.             err=dl_relocate(pHandle, &rel[j], addr);
133.             if(err!=DL_NO_ERR){
134.                 goto fail1;
135.             }
136.         }
137.         else { //从外部导入符号
138.             //由于编译器原因暂不支持
139.             DL_LOG_E("Module: can't find %s in kernel symbol table", strtab + sym->st_name);
140.             err=DL_UNDEF_SYM_ERR;
141.             goto fail1;
142.         }
143.     }
144. }
145.
146. //构建符号表

```

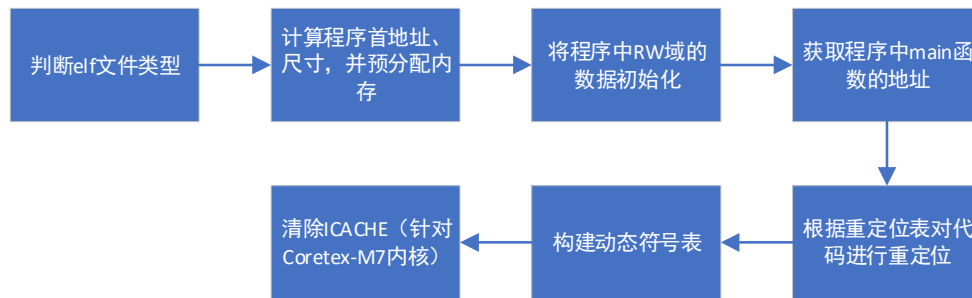
```
147.     for(dynsym_off=0;dynsym_off<elf_header(lib_ptr)->e_shnum;dynsym_off++){
148.         uint8_t *shstrab;
149.         shstrab = (uint8_t *)lib_ptr +
150.             elf_shdr(lib_ptr)[elf_header(lib_ptr)->e_shstrndx].sh_offset;
151.         if(strcmp((const char *) (shstrab + elf_shdr(lib_ptr)[dynsym_off].sh_name),
            ELF_DYNSYM)==0)
152.             break;
153.     }
154.
155.     if(dynsym_off!=elf_header(lib_ptr)->e_shnum){
156.         int count = 0;
157.         Elf_Sym *symtab = NULL;
158.         uint8_t *strtab = NULL;
159.
160.         symtab = (Elf_Sym *)((uint8_t *)lib_ptr + elf_shdr(lib_ptr)[dynsym_off].sh_
            offset);
161.         strtab = (uint8_t *)lib_ptr + elf_shdr(lib_ptr)[elf_shdr(lib_ptr)[dynsym_of
            f].sh_link].sh_offset;
162.
163.         //查找动态段符号个数，方便预分配内存
164.         for (size_t i = 0; i < elf_shdr(lib_ptr)[dynsym_off].sh_size / sizeof(Elf_S
            ym); i++){
165.             if ((ELF_ST_BIND(symtab[i].st_info) == STB_GLOBAL) &&
166.                 (ELF_ST_TYPE(symtab[i].st_info) == STT_FUNC))
167.                 count ++;
168.         }
169.
170.         pHandle->nsym = count;
171.         pHandle->symtab = (DL_Symbol_Type *)dl_port_malloc(count*sizeof(DL_Symbol_T
            ype));
172.         if(pHandle->nsym != 0){
173.             if(pHandle->symtab == NULL){
174.                 DL_LOG_E("Module: allocate space failed.");
175.                 err=DL_MALLOC_FAIL;
176.                 goto fail2;
177.             }
178.             memset(pHandle->symtab,0,count*sizeof(DL_Symbol_Type));
179.
180.             //将动态符号段的内容复制到内存中
181.             for (size_t i = 0, j = 0; i < elf_shdr(lib_ptr)[dynsym_off].sh_size / s
                izeof(Elf_Sym); i++){
182.                 size_t length;
183.                 //仅保留全局函数类型的符号
```

```

184.         if ((ELF_ST_BIND(symtab[i].st_info) != STB_GLOBAL) ||
185.             (ELF_ST_TYPE(symtab[i].st_info) != STT_FUNC))
186.             continue;
187.
188.         length = strlen((const char *)(strtab + symtab[i].st_name)) + 1;
189.
190.         pHandle->symtab[j].addr =
191.             (void *)((uint8_t*)pHandle->mem_ptr + symtab[i].st_value - pHandle->vstart_addr);
192.         pHandle->symtab[j].name = dl_port_malloc(length);
193.         if(pHandle->symtab[j].name==NULL){
194.             DL_LOG_E("Module: allocate space failed.");
195.             err=DL_MALLOC_FAIL;
196.             goto fail3;
197.         }
198.         memcpy((void *)pHandle->symtab[j].name,
199.             strtab + symtab[i].st_name,
200.             length);
201.         ((char*)pHandle->symtab[j].name)[length-1]='\0';
202.         j ++;
203.     }
204. }
205. }
206. else{
207.     pHandle->symtab=NULL;
208.     pHandle->nsym=0;
209. }
210. #if defined(DL_CACHE_USE)&& DL_CACHE_USE==1
211.     dl_port_cache_process(pHandle->mem_ptr,pHandle->mem_size);
212. #endif
213.     return err;
214. fail3:
215.     for(size_t i = 0;i < pHandle->nsym;i++){
216.         dl_port_free((void*)pHandle->symtab[i].name);
217.     }
218. fail2:
219.     dl_port_free(pHandle->symtab);
220. fail1:
221.     dl_port_aligned_free(pHandle->mem_ptr);
222. fail0:
223.     return err;
224. }

```

函数执行过程大致如下：



1、判断 elf 文件类型

在代码的第 13~26 行用于判断 elf 文件是否是我们实现的动态加载器支持的类型。其中 `ELFMAG` 是 elf 文件的识别标识，用于指示该文件为 elf 文件。第 18~21 行指示动态加载器目前仅支持 32 位平台编译的 elf 文件。第 23~26 行限制加载的 elf 文件只能是共享库类型和可执行的 elf 文件类型，为什么会多一个可执行的 elf 文件类型呢？因为 MDK 编译输出的 .axf 就是可执行的 elf 文件类型。

2、计算程序首地址、尺寸，并预分配内存

在代码的第 30~87 行进行了这个操作。所谓的计算首地址是这样的，在我们编译生成 .axf 文件时，选择了一个 .sct 后缀的文件，这个文件也叫做分散加载文件，用于告诉链接器如何分配代码和数据的物理地址。上一章写的示例 .sct 文件中，所有域的起始地址被设置为了 0，这个起始地址就是所谓的首地址。在一些其他的实现中，这个首地址可能不是 0，显然首地址前面的内存是没有用的，为了节省程序镜像所占有的内存，我们仅分配首地址后面的内存，并在 .axf 文件中所有出现与地址有关的量都减去首地址。

注意代码中的第 81 行，这里内存分配函数使用的是 `dl_port_aligned_alloc` 函数对整个程序镜像进行分配，这个函数是用来分配首地址按指定字节数对齐的一片内存的，在 elf 文件中，每个域可能都会有着不同的对齐需求，查看生成的 elf 文件每个域的对齐要求可以使用 `fromelf` 查看，查看的命令为：

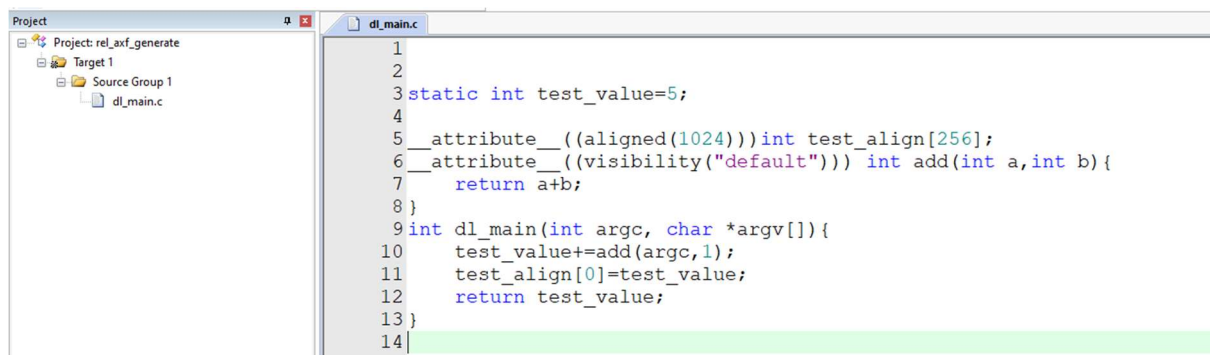
`fromelf rel_axf_generate.axf`

```
** Section #1 'ER_IROM1' (SHT_PROGBITS) [SHF_ALLOC + SHF_EXECINSTR]
   Size   : 44 bytes (alignment 8)
   Address: 0x00000000

** Section #2 'RW_IRAM1' (SHT_PROGBITS) [SHF_ALLOC + SHF_WRITE]
   Size   : 4 bytes (alignment 4)
   Address: 0x00000400

** Section #3 'ZI_IRAM2' (SHT_NOBITS) [SHF_ALLOC + SHF_WRITE]
   Size   : 1024 bytes (alignment 1024)
   Address: 0x00000800
```

测试代码:



```
1
2
3 static int test_value=5;
4
5 __attribute__((aligned(1024))) int test_align[256];
6 __attribute__((visibility("default"))) int add(int a,int b){
7     return a+b;
8 }
9 int dl_main(int argc, char *argv[]){
10     test_value+=add(argc,1);
11     test_align[0]=test_value;
12     return test_value;
13 }
14
```

这里我们只需要关注这三个节区（ER_IROM1、RW_IRAM1、ZI_IRAM2）的对齐参数，加载的时候只加载这三个节区，这三个节区的名字与我们编写的.sct 文件中域的名字是一一对应的，ER_IROM1 存放 RO 类型与代码类型的数据，RW_IRAM1 对应 RW 域，ZI_IRAM2 对应 ZI 域。

代码中 test_value 被分配到了 RW 域，test_align 被分配到了 ZI 域，并指定其为 1024 字节对齐，因此我们可以看到，RW_IRAM1 的对齐要求是 4 字节，ZI_IRAM2 的对齐要求为 1024 字节。

3、将程序中 RW 域中的数据进行初始化

这一步在代码第 89~96 行，比较简单，简单的复制。

4、获取程序中 main 函数的地址

这一步在代码第 98~104 行，同样比较简单。

5、根据重定位表对代码进行重定位

这一步在代码的第 106~144 行，是我们实现动态加载的关键。程序首先遍历了所有带有重定位属性的节区，然后再遍历每个节区的重定位表。函数 dl_relocate 用来进行重定位操作。该函数原型如下：

1. //实现根据特定指令，对机器码进行相应修改，每种芯片架构的方法都不同
2. //pHandle: 句柄
3. //rel: 重定位表，包含要更改的机器码的相对地址以及更改的方法描述
4. //sym_addr: 符号地址，包含更改的内容
5. //返回值: DL_Err_Type 错误码
6. DL_Err_Type dl_relocate(DL_Handler *pHandle, Elf32_Rel *rel, Elf32_Addr sym_addr)

该函数的第一个输入参数 pHandle 是输入句柄，第二个参数是重定位表项，这个表项包含偏移地址以及需要重定位的操作，换句话说就是这一项告诉了我们，这个代码哪中里需要根据内存位置修改一下，以什么方法去修改。第三个参数就是我们需要修改的

内容。这个函数代表了汇编与机器码的对应关系，与芯片架构密切相关。不同的芯片指令构成不同，该函数的实现也不同，是实现重定位的难点所在，我目前仅实现了 arm 芯片的重定位操作，下面简要介绍下我是如何实现这个函数的。

函数定义如下：

```
1  /*
2  *   @File name: dl_arch.c
3  *   @Author: wzh
4  *   @Version: 1.0
5  *   @Date: 2021-12-3
6  *   @Description: 单片机程序动态加载的平台文件，主要实现重定位时，对机器码的更改
7  *   @Todo    目前仅支持arm平台
8  */
9  #include "dl_lib.h"
10 #include "dl_elf.h"
11 #include "dl_port.h"
12
13 #if defined(__arm__) && DL_USE_ARCH==DL_ARCH_ARM
14 //根据arm elf 文件规范编写
15 //部分参考了RT-Thread的Moudle实现
16 //arm elf 文件规范文档网址
17 //https://developer.arm.com/documentation/ihi0044/h/?lang=en#dynamic-linking
18 /*
19  * Relocation type for arm
20  */
21 #define R_ARM_NONE                0
22 #define R_ARM_PC24                1
23 #define R_ARM_ABS32               2
24 #define R_ARM_REL32               3
25 #define R_ARM_LDR_PC_G0           4
26 #define R_ARM_ABS16               5
27 #define R_ARM_ABS12               6
28 #define R_ARM_THM_ABS5            7
29 #define R_ARM_ABS8                8
30 #define R_ARM_SBREL32             9
31 #define R_ARM_THM_CALL            10
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76 //实现根据特定指令，对机器码进行相应修改，每种芯片架构的方法都不同
77 //pHandle: 句柄
78 //rel: 重定位表，包含要更改的机器码的相对地址以及更改的方法描述
79 //sym_addr: 符号地址，包含更改的内容
80 //返回值: DL_Err_Type 错误码
81 DL_Err_Type dl_relocate(DL_Handler *pHandle, Elf32_Rel *rel, Elf32_Addr sym_addr){
82     Elf32_Addr *where;
83
84     where = (Elf32_Addr *) ((uint8_t *)pHandle->mem_ptr
85                             + rel->r_offset
86                             - pHandle->vstart_addr);
87
88     //where 要更改的地址
89     //sym_addr 符号的地址
90     switch (ELF32_R_TYPE(rel->r_info))
91     {
92     case R_ARM_NONE:
93         break;
94     case R_ARM_ABS32:
95         *where += (Elf32_Addr)sym_addr;
96         DL_LOG_D("R_ARM_ABS32: %x -> %x",
97                 (uint32_t)where, *where);
98         break;
99     }
```

关于每个重定位符号的意义，可以参考 arm 官网上关于动态加载方面的叙述，原来的链接没了，大家可以再找找，其中有关于重定位符号的详细叙述。

经过测试，在 MDK 编译的工具链中，R_ARM_THM_MOVW_ABS_NC 与

R_ARM_THM_MOVT_ABS 用的最多，分别对应于汇编指令 MOVW 与 MOVT 指令。

```
case R_ARM_THM_MOVT_ABS_NC: {
    uint32_t tmp;
    DL_LOG_D("MOVW_ABS_NC:sym_addr:0x%08x,*sym_addr:0x%08x", sym_addr, *(uint32_t*)sym_addr);
    tmp = *(uint32_t*)where;
    tmp = (tmp >> 16) | (tmp << 16); // 高低位互换
    tmp &= 0xFBF08F00; // 对原址[19:16],[26],[14:12],[7:0]清0
    sym_addr &= 0x0000ffff; // 高16位清0, MOVW指令仅支持16位立即数
    if (ELF_ST_TYPE(rel->r_info) == STT_FUNC) // 确保代码跳转处于Thumb状态
        sym_addr |= 1;
    tmp |= ((sym_addr >> 12) & 0xf) << 16; // tmp[19:16] = (sym_addr >> 12) & 0xf
    tmp |= ((sym_addr >> 11) & 0x1) << 26; // tmp[26] = (sym_addr >> 11) & 0x1
    tmp |= ((sym_addr >> 8) & 0x7) << 12; // tmp[14:12] = (sym_addr >> 8) & 0x7;
    tmp |= sym_addr & 0xff; // tmp[7:0] = sym_addr & 0xff
    tmp = (tmp >> 16) | (tmp << 16); // 高低位互换
    DL_LOG_D("MOVW_ABS_NC:where:%p,*where:0x%08x,tmp:%08x", where, *where, tmp);
    *(uint32_t*)where = tmp;
}

break;
case R_ARM_THM_MOVT_ABS: {
    uint32_t tmp;
    DL_LOG_D("R_ARM_THM_MOVT_ABS:sym_addr:0x%08x,*sym_addr:0x%08x", sym_addr, *(uint32_t*)sym_addr);
    tmp = *(uint32_t*)where;
    tmp = (tmp >> 16) | (tmp << 16); // 高低位互换
    tmp &= 0xFBF08F00; // 对原址[19:16],[26],[14:12],[7:0]清0
    sym_addr &= 0xffff0000; // 低16位清0, MOVT指令仅支持16位立即数
    tmp |= ((sym_addr >> 28) & 0xf) << 16; // tmp[19:16] = (sym_addr >> 28) & 0xf
    tmp |= ((sym_addr >> 27) & 0x1) << 26; // tmp[26] = (sym_addr >> 17) & 0x1
    tmp |= ((sym_addr >> 24) & 0x7) << 12; // tmp[14:12] = (sym_addr >> 24) & 0x7;
    tmp |= (sym_addr >> 16) & 0xff; // tmp[7:0] = (sym_addr >> 16) & 0xff
    tmp = (tmp >> 16) | (tmp << 16); // 高低位互换
    DL_LOG_D("MOVT_ABS:where:%p,*where:0x%08x,tmp:%08x", where, *where, tmp);
    *(uint32_t*)where = tmp;
}
}
```

6、构建动态符号表

代码第 146 行~209 行，此处就是将 .axf 文件中导出的符号，以字符串-函数地址的方式添加到句柄中。

7、清除 Cache

代码第 210~212 行，假如使用的是 Cortex-M7 内核的芯片，需要在这里清除一下 ICAHE，否则会有有一定的概率在执行跳转程序镜像时出现死机的现象。

4.3.axf 文件调用被加载器函数的方法介绍

通过动态加载器以及带重定位节区 .axf 文件，我们实现了加载器一方对 .axf 文件内所包含函数的调用，但有很多场景，需要 .axf 文件调用加载器中的一些函数，假如我们想要输出 “hello world”，我们使用现有的程序就不容易办到，这里给出两种方法实现。

4.3.1 通过固定的函数表访问

这个思路比较简单，就是将二者使用的公共函数放到一块固定的内存地址中，.axf 文件通过访问这个函数表获取函数指针，进而直接访问。这种办法简单易行，但是需要编程者合理安排这个函数表的位置。

4.3.2 通过特定的指令访问

这个方法适用于使用 RTX 操作系统，在 RTX 中，可以通过 SVC 指令获取函数指针，具体如下：

```
1  #ifndef DL_EXTERN_LIB_H_
2  #define DL_EXTERN_LIB_H_
3
4  #include <stdio.h>
5  #include <stdarg.h>
6  #include <stdlib.h>
7  #include <stdint.h>
8  #include <time.h>
9  #include <string.h>
10
11 #attribute__((always_inline)) static inline void* dl_vector_get_lib_func_entry(size_t index){
12     register size_t r0 __asm("r0") = (size_t)index;
13     __asm volatile ("svc 8": "r"(__r0): "r"(__r0): "cc", "memory");
14     return (void*)r0;
15 }
16
17 #define DL_STATIC_ASSERT(exp) typedef int __ai[(exp)?1:-1]
18
19 #define DL_FUNC_NAME(name) dl_use_##name
20 #define DL_DECLARE_EXTERN_LIB_FUNC(func, base, max_func_num, index, ret, ...) \
21     ret (*DL_FUNC_NAME(func))(__VA_ARGS__)=dl_vector_get_lib_func_entry(index+base);\
22     DL_STATIC_ASSERT(index<max_func_num)
23
```

这里我规定 svc 8 为所有 .axf 程序获取加载函数指针的函数，调用这个指令后，会执行到 dl_vector.c 的这个函数：

```
88     STRING_LIB_DECLARE(strxfrm, STRXFRM)
89 };
90
91 void* svc_dl_vector_get_lib_func_entry(size_t index){
92     if(index<sizeof(DL_Vector_Table)/sizeof(DL_Vector_Table[0])){
93         return DL_Vector_Table[index];
94     }
95     else
96         return NULL;
97 }
98
99
```

这个函数会根据 index，返回对应的函数指针。这样就实现了被加载的 .axf 文件，调用动态加载器方函数的过程。

4.4 总结

至此，STM32 的动态加载实现就全部讲解完毕了，实际使用中还需要多多练习，适应这种编程思想，关于动态加载器的实现如果我又哪里讲的不太清晰的，或者在实际使用中遇到 BUG 了，都可以在论坛帖子下方联系我 。

帖子链接：<https://www.armbbs.cn/forum.php?mod=viewthread&tid=109952&extra=>